Kubernetes

Run workloads on Kubernetes from architecture and networking to Helm, autoscaling, operators, cluster upgrades, RBAC, troubleshooting and exam strategy.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Kubernetes architecture (15 min)
2. Pods, Deployments and probes (17 min)
3. Services, DNS and Ingress (16 min)
4. ConfigMaps and Secrets (14 min)
5. Storage and StatefulSets (15 min)
6. Jobs, CronJobs, DaemonSets and StatefulSet patterns (15 min)
7. Resource requests, limits, QoS and quotas (16 min)
8. Autoscaling: HPA, VPA, KEDA and the cluster autoscaler (15 min)
9. RBAC and workload security (17 min)
10. Scheduling and troubleshooting (18 min)
11. Helm and Kustomize (16 min)
12. Custom resources, controllers and operators (15 min)
13. CNI, Pod networking and network policies (16 min)
14. Building, upgrading and backing up a cluster (18 min)
15. CKA and CKS exam strategy (14 min)
16. Probes, metrics and logging on Kubernetes (15 min)
17. Service mesh, mutual TLS and traffic control (14 min)
18. Network policies in practice (13 min)
19. StatefulSets and stateful workloads (14 min)
20. Debugging Kubernetes workloads (18 min)
Cheat sheet

Lesson 1. Kubernetes architecture

15 minutes. Learn the control plane and node components, the declarative reconcile loop, and how to read cluster state with kubectl.

You run one container on one server and it works. Now you need ten copies across five servers, automatic replacement when one dies, and updates without downtime. Doing that by hand does not scale. Kubernetes is the system that does it for you: you describe what you want, and it keeps working until reality matches.

What you will learn

- The components of the control plane and of a worker node
- The declarative model and the reconcile loop
- The core objects you will meet in every cluster
- The first kubectl commands to run on an unfamiliar cluster

The components

A cluster is a set of machines called **nodes**. The **control plane** makes decisions; **worker nodes** run your containers.

Component	Where	Job
`kube-apiserver`	Control plane	The front door. Every client and component talks to it. It validates and stores requests
`etcd`	Control plane	The database holding all cluster state. Back it up
`kube-scheduler`	Control plane	Chooses a node for each new Pod
`kube-controller-manager`	Control plane	Runs controllers that drive actual state toward desired state
`kubelet`	Every node	Starts and monitors the containers assigned to its node
`kube-proxy`	Every node	Maintains network rules so Services reach Pods
Container runtime	Every node	Actually runs containers (containerd or CRI-O)

Declarative and reconciling

You do not tell Kubernetes *how* to do something. You submit a **desired state**, for example "run 3 replicas of this image", and controllers repeatedly compare it with the **actual state** and act on the difference. If a node dies and a replica disappears, the controller notices 2 is not 3 and creates a replacement. This loop is why Kubernetes heals itself, and it is the most important idea in this module.

```
$ kubectl create deployment web --image=nginx:1.27 --replicas=3
deployment.apps/web created
$ kubectl get pods -l app=web
NAME                   READY   STATUS    RESTARTS   AGE
web-7c5ddbdf54-4xk2p   1/1     Running   0          20s
web-7c5ddbdf54-9tq8m   1/1     Running   0          20s
web-7c5ddbdf54-zl6vh   1/1     Running   0          20s
$ kubectl delete pod web-7c5ddbdf54-4xk2p
pod "web-7c5ddbdf54-4xk2p" deleted
$ kubectl get pods -l app=web
```

After the delete, run the last command again and you will see three Pods once more, one of them brand new. You asked for 3, so Kubernetes made it so.

MARGA by EknathaLabs
marga.eknathalabs.com

Core objects

Object	Purpose
Pod	One or more containers sharing a network and storage. The smallest unit you deploy
Deployment	Manages identical Pods, rolling updates and rollbacks
Service	A stable name and IP in front of a changing set of Pods
ConfigMap / Secret	Configuration and sensitive data, kept out of images
Namespace	A named partition of the cluster for teams or environments
Node	A machine in the cluster

Objects are written as YAML with four top-level parts: `apiVersion`, `kind`, `metadata` and `spec`. You will write many in the coming lessons.

First commands on any cluster

```
$ kubectl config current-context
prod-eu-west
$ kubectl get nodes -o wide
$ kubectl get pods -A
$ kubectl describe node worker-2 | tail -15
$ kubectl api-resources | head
$ kubectl explain pod.spec.containers
```

Always check `current-context` first. Running a command against the wrong cluster is a classic and expensive mistake. `kubectl explain` documents any field from inside the terminal, which is also what you will rely on in an exam.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Kubernetes architecture fits together

Part	What it does	What goes wrong
The components	A cluster is a set of machines called nodes.	Editing Pods directly instead of the Deployment that owns them.
Declarative and reconciling	You do not tell Kubernetes how to do something.	Declarative and reconciling You do not tell Kubernetes how to do something.
Core objects	Objects are written as YAML with four top-level parts: apiVersion, kind, metadata and spec.	Forgetting the namespace (-n) and concluding that a resource is missing.
First commands on any cluster	Always check current-context first. Running a command against the wrong cluster is a classic and expensive mistake.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: The components to Declarative and reconciling / Declarative and reconciling to Core objects / Core objects to First commands on any cluster

Incident: kubectl is pointed at the wrong cluster

> **Incident:** *You run* `kubectl delete deployment web` *to clean up a test, and an alert fires: the production website is down.*

Your shell was still using the production context from earlier. Confirm and prevent:

```
$ kubectl config get-contexts
CURRENT   NAME          CLUSTER       NAMESPACE
*         prod-eu-west  prod-eu-west  shop
          dev-sandbox   dev-sandbox   default
$ kubectl config use-context dev-sandbox
```

Restore service first by re-applying the Deployment manifest from your records (`kubectl apply -f web.yaml` on the production context), then fix the habit: show the context in your shell prompt, use a separate kubeconfig file per environment, and add `--context` explicitly in scripts. Role-based access (a later lesson) should also stop everyday accounts from deleting production workloads at all.

Common mistakes

- Editing Pods directly instead of the Deployment that owns them
- Forgetting the namespace (`-n`) and concluding that a resource is missing
- Treating Pod IP addresses as stable
- Not backing up etcd on self-managed clusters

Check yourself

Which component stores all cluster state?

Answer: etcd

You delete one Pod that belongs to a Deployment with 3 replicas. What happens?

Answer: A controller notices the difference and creates a replacement Pod

Which component decides which node a new Pod runs on?

Answer: kube-scheduler

Which command shows the documentation of a field such as pod.spec.containers?

Answer: kubectl explain pod.spec.containers

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Pods, Deployments and probes

17 minutes. Write a production-shaped Deployment with resource requests, health probes and a safe rolling update, and roll it back when it fails.

A Deployment that merely "runs the container" is a Deployment that will hurt you. It will receive traffic before it is ready, hog or starve its neighbours, and roll out a broken version to every replica. Resource settings, probes and an update strategy turn a demo into something you can operate.

What you will learn

- The anatomy of a Pod and a Deployment
- Requests and limits, and what each does
- Readiness, liveness and startup probes
- Rolling updates, rollbacks and rollout status

A production-shaped Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
  namespace: shop
spec:
  replicas: 3
  selector:
    matchLabels:
      app: shop-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: api
          image: registry.example.com/shop-api:1.8.2
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /healthz/ready
              port: 8080
            periodSeconds: 5
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /healthz/live
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 10
```

The `selector` must match the Pod template's labels; that link is how the Deployment knows which Pods it owns. Apply it with `kubectl apply -f shop-api.yaml`.

MARGA by EknathaLabs
marga.eknathalabs.com

Requests and limits

Setting	Meaning	Effect
`requests`	What the container is guaranteed	The scheduler uses it to place the Pod
`limits.memory`	A hard ceiling	Exceeding it gets the container killed (OOMKilled)
`limits.cpu`	A throttle	Exceeding it slows the container; it is not killed

Set a memory request near typical use and a memory limit with headroom. Many teams omit CPU limits to avoid throttling while still setting CPU requests. Without requests the scheduler is guessing, and nodes become overcommitted.

Probes

Probes let Kubernetes ask your application how it is doing. They answer three different questions:

Probe	Question	Failure action
Readiness	Can you take traffic right now?	Pod removed from Service endpoints, not restarted
Liveness	Are you stuck beyond recovery?	Container restarted
Startup	Have you finished starting?	Holds off the other probes until it passes

> **Warning:** *Do not make a liveness probe depend on a database or another service. If the database is slow, every Pod fails its liveness check, all restart together, and a partial slowdown becomes a full outage. Liveness should only check the process itself.*

Rolling updates and rollbacks

With `maxUnavailable: 0` and `maxSurge: 1`, Kubernetes starts one new Pod, waits until it is **ready**, then removes one old Pod, and repeats. Readiness probes are therefore what stop a broken version from replacing a working one.

```
$ kubectl set image deployment/shop-api api=registry.example.com/shop-api:1.9.0 -n shop
$ kubectl rollout status deployment/shop-api -n shop
Waiting for deployment "shop-api" rollout to finish: 1 out of 3 new replicas have been updated...
$ kubectl rollout history deployment/shop-api -n shop
$ kubectl rollout undo deployment/shop-api -n shop
deployment.apps/shop-api rolled back
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Pods, Deployments and probes fits together

Part	What it does	What goes wrong
A production-shaped Deployment	The selector must match the Pod template's labels; that link is how the Deployment knows which Pods it owns.	Selector and template labels that do not match.
Requests and limits	Set a memory request near typical use and a memory limit with headroom.	No memory limit, or a limit equal to average use.
Probes	Probes let Kubernetes ask your application how it is doing.	No readiness probe, so traffic arrives before the app can handle it.
Rolling updates and rollbacks	With maxUnavailable: 0 and maxSurge: 1, Kubernetes starts one new Pod, waits until it is ready, then removes one old Pod, and repeats.	Liveness probes that check dependencies.

Flow: A production-shaped Deployment to Requests and limits / Requests and limits to Probes / Probes to Rolling updates and rollbacks

MARGA by EknathaLabs marga.eknathalabs.com

Incident: a bad release stalls halfway

Incident: *After deploying 1.9.0,* `rollout status` *hangs. Users are fine, but the release never finishes.*

```
$ kubectl get pods -n shop -l app=shop-api
NAME                        READY   STATUS    RESTARTS   AGE
shop-api-6d8f7c9b5-aaaaa    1/1     Running   0          2d
shop-api-6d8f7c9b5-bbbbb    1/1     Running   0          2d
shop-api-6d8f7c9b5-ccccc    1/1     Running   0          2d
shop-api-84c6b7d4f-ddddd    0/1     Running   0          4m
$ kubectl describe pod shop-api-84c6b7d4f-ddddd -n shop | tail -5
  Warning  Unhealthy  3m  kubelet  Readiness probe failed: HTTP probe failed with statuscode: 503
```

The new Pod runs but never becomes ready, so the rollout cannot continue. This is the strategy working as designed: the three healthy old Pods keep serving and users noticed nothing. Look at its logs (`kubectl logs shop-api-84c6b7d4f-ddddd -n shop`), find the cause, then either ship a fix or abandon the release with `kubectl rollout undo`. After a failed attempt, `kubectl rollout status` returns an error once the progress deadline passes, which your pipeline can use to fail the deploy automatically.

Predict the rollout

Common mistakes

- No readiness probe, so traffic arrives before the app can handle it
- Liveness probes that check dependencies
- No memory limit, or a limit equal to average use
- Using `latest` tags so a rollback cannot go back to anything
- Selector and template labels that do not match

Check yourself

A new Pod fails its readiness probe. What does Kubernetes do?

Answer: Stops sending it traffic and, in a rolling update, does not continue the rollout

Which setting makes a container get OOMKilled when exceeded?

Answer: limits.memory

Why should a liveness probe not check the database?

Answer: A database problem would make every Pod restart at once and worsen the outage

Which command reverts a Deployment to its previous revision?

Answer: kubectl rollout undo deployment/shop-api

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Services, DNS and Ingress

16 minutes. Give changing Pods a stable address with Services, understand cluster DNS, expose apps with Ingress and restrict traffic with NetworkPolicy.

Pods are disposable. Each replacement gets a new IP address, so nothing can rely on one. A **Service** solves that by providing a stable name and virtual IP in front of whichever Pods currently match. Most "it cannot connect" problems in Kubernetes come down to Services that select nothing.

What you will learn

- The Service types and when to use each
- How labels connect a Service to Pods
- Cluster DNS names
- Ingress for HTTP routing, and NetworkPolicy for restriction

Services

```
apiVersion: v1
kind: Service
metadata:
  name: shop-api
  namespace: shop
spec:
  selector:
    app: shop-api
  ports:
    - port: 80
      targetPort: 8080
```

The Service sends traffic arriving on port 80 to port 8080 of every **ready** Pod that has the label `app: shop-api`. The list of those Pod addresses is called the Service's **endpoints**.

Type	Reachable from	Typical use
`ClusterIP` (default)	Inside the cluster	Internal service-to-service calls
`NodePort`	Any node's IP on a high port	Simple tests, on-premises setups
`LoadBalancer`	A cloud load balancer's address	Exposing one service directly
`ExternalName`	Inside the cluster, as a DNS alias	Pointing at an external hostname

Cluster DNS

Every Service gets a DNS name: `<service>.<namespace>.svc.cluster.local`. Within the same namespace the short name `shop-api` works; from another namespace use `shop-api.shop`.

```
$ kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -n shop -- nslookup shop-api
Name:      shop-api.shop.svc.cluster.local
Address 1: 10.96.142.7
```

This throwaway Pod is the standard way to test connectivity from inside the cluster.

Ingress

Creating a `LoadBalancer` per service gets expensive. An **Ingress** routes HTTP and HTTPS traffic from one entry point to many Services by host or path. It needs an **Ingress controller** (for example ingress-nginx) running in the cluster; the Ingress object alone does nothing.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
spec:
  ingressClassName: nginx
  tls:
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    - hosts: [shop.example.com]
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: shop-api
                port:
                  number: 80
```

NetworkPolicy

By default every Pod can talk to every other Pod. A NetworkPolicy restricts that, but only if your network plugin enforces policies. A common pattern is to deny everything and then allow what is needed:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-api-only
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app: postgres
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: shop-api
      ports:
        - port: 5432
```

Incident: the Service has no endpoints

> **Incident:** `curl http://shop-api` *from another Pod times out or is refused, yet* `kubectl get pods` *shows all three Pods running.*

```
$ kubectl get endpoints shop-api -n shop
NAME       ENDPOINTS   AGE
shop-api   <none>      9m
$ kubectl get svc shop-api -n shop -o jsonpath='{.spec.selector}'
{"app":"shop-api"}
$ kubectl get pods -n shop --show-labels
NAME                        READY   STATUS    LABELS
shop-api-84c6b7d4f-ddddd    1/1     Running   app=shop-backend,pod-template-hash=84c6b7d4f
```

`<none>` is the clue. The Service selects `app=shop-api`, but the Pods carry `app=shop-backend`, so nothing matches. Fix the label in the Deployment template or the selector in the Service so they agree, apply, and check that the endpoints list appears. A second cause of empty endpoints is Pods that are running but not **ready**, so always look at both labels and the READY column. A third is the wrong `targetPort`, which gives you endpoints but connection refused.

Diagram: How a request reaches a pod

Part	What it does	What goes wrong
Client	Sends HTTPS to the public name.	A wrong DNS record sends it elsewhere.
Load balancer	Forwards to the ingress controller.	If its health check fails, no traffic passes.
Ingress	Matches host and path and picks a Service.	A wrong path rule returns 404 from the controller.
Service	Gives a stable address and selects pods by label.	A selector that matches no pods gives empty endpoints.
Pod	Receives the request on the container port.	A wrong targetPort gives connection refused.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Client to Load balancer / Load balancer to Ingress / Ingress to Service / Service to Pod

Common mistakes

- Mismatched labels between Service selector and Pods
- Using the Pod port as `port` and the Service port as `targetPort` the wrong way round
- Creating an Ingress without an Ingress controller
- Writing NetworkPolicies on a network plugin that does not enforce them

Check yourself

A Service shows ENDPOINTS <none>. What is the most likely cause?

Answer: The selector does not match any ready Pod's labels

What is the full DNS name format of a Service?

Answer: service.namespace.svc.cluster.local

What does an Ingress require to work?

Answer: An Ingress controller running in the cluster

In a Service, what does targetPort refer to?

Answer: The port the container listens on

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. ConfigMaps and Secrets

14 minutes. Separate configuration from images with ConfigMaps and Secrets, choose between environment variables and mounted files, and handle secrets honestly.

One image should run unchanged in development, staging and production. What changes between them is configuration: database hostnames, feature flags, credentials. Kubernetes keeps that configuration in separate objects and injects it when the Pod starts.

What you will learn

- ConfigMaps for non-sensitive settings
- Secrets for sensitive values, and their real limits
- Injecting as environment variables versus mounted files
- What happens when configuration changes

ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-config
  namespace: shop
data:
  LOG_LEVEL: info
  FEATURE_NEW_CHECKOUT: "true"
  app.properties: |
    cache.ttl=300
    region=eu-west
```

Values are strings, so booleans and numbers are quoted. Keys can hold single values or whole files.

Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: shop-db
  namespace: shop
type: Opaque
stringData:
  username: app
  password: change-me
```

`stringData` accepts plain text and Kubernetes stores it Base64-encoded in `data`.

> **Warning:** *Base64 is an encoding, not encryption. Anyone who can read the Secret object can decode it with* `base64 -d`*. Real protection comes from three things: RBAC that limits who can read Secrets, encryption at rest for etcd, and ideally an external secret manager that syncs values in. Never commit Secret manifests containing real values to a shared repository.*

MARGA by EknathaLabs
marga.eknathalabs.com

Using them in a Pod

```
spec:
  containers:
    - name: api
      image: registry.example.com/shop-api:1.8.2
      env:
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: shop-config
              key: LOG_LEVEL
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: shop-db
              key: password
      volumeMounts:
        - name: config
          mountPath: /etc/shop
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: shop-config
        items:
          - key: app.properties
            path: app.properties
```

Method	Updates when the object changes?	Notes
Environment variable	No, only at Pod start	Simple; values appear in process environment
Mounted volume	Yes, files refresh after a short delay	The application must re-read the file

Mounted files are usually the better choice for secrets, because environment variables leak easily into crash dumps and debug output.

Verifying what a Pod sees

```
$ kubectl exec deploy/shop-api -n shop -- printenv LOG_LEVEL
info
$ kubectl exec deploy/shop-api -n shop -- cat /etc/shop/app.properties
cache.ttl=300
region=eu-west
$ kubectl get secret shop-db -n shop -o jsonpath='{.data.password}' | base64 -d
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How ConfigMaps and Secrets fits together

Part	What it does	What goes wrong
ConfigMap	Values are strings, so booleans and numbers are quoted.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Secret	stringData accepts plain text and Kubernetes stores it Base64-encoded in data.	Putting secrets in images, ConfigMaps or Deployment manifests.
Using them in a Pod	Mounted files are usually the better choice for secrets, because environment variables leak easily into crash dumps and debug output.	Expecting environment variables to refresh without a restart.
Verifying what a Pod sees	.	Treating Base64 as secure.

Flow: ConfigMap to Secret / Secret to Using them in a Pod / Using them in a Pod to Verifying what a Pod sees

MARGA by EknathaLabs marga.eknathalabs.com

Incident: changing the ConfigMap did nothing

> **Incident:** *You change* `LOG_LEVEL` *from* `info` *to* `debug` *in the ConfigMap and apply it. The application keeps logging at* `info`*.*

```
$ kubectl get configmap shop-config -n shop -o jsonpath='{.data.LOG_LEVEL}'
debug
$ kubectl exec deploy/shop-api -n shop -- printenv LOG_LEVEL
info
```

The object is updated, but the running Pods still hold the old value. Environment variables are read once when the container starts. Restart the Pods to pick up the change:

```
$ kubectl rollout restart deployment/shop-api -n shop
$ kubectl rollout status deployment/shop-api -n shop
```

For a durable approach, many teams include a hash of the config in the Pod template (Helm and Kustomize can do this), so any config change automatically triggers a rolling update and the change is recorded as a new revision you can roll back.

Common mistakes

- Treating Base64 as secure
- Expecting environment variables to refresh without a restart
- Putting secrets in images, ConfigMaps or Deployment manifests
- Naming a key wrongly, which leaves the Pod stuck in `CreateContainerConfigError`

Check yourself

Is a Kubernetes Secret encrypted by default?

Answer: No, values are only Base64-encoded unless encryption at rest and RBAC are configured

You update a ConfigMap used through environment variables. When do running Pods see the change?

Answer: Only after the Pods are restarted

Which injection method can refresh while the Pod is running?

Answer: A mounted ConfigMap or Secret volume

A Pod is stuck in CreateContainerConfigError. What should you check first?

Answer: That the referenced ConfigMap or Secret and its keys exist

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Storage and StatefulSets

15 minutes. Persist data with PersistentVolumeClaims and StorageClasses, and know when a StatefulSet is the right controller.

Containers are disposable, but a database is not. When a Pod running PostgreSQL is rescheduled to another node, its data must follow it. Kubernetes separates the *request* for storage from the *supply* of storage, so applications do not need to know where disks come from.

What you will learn

- PersistentVolume, PersistentVolumeClaim and StorageClass
- Access modes and reclaim policies
- When to use a StatefulSet instead of a Deployment
- How to diagnose a Pod stuck waiting for a volume

The three objects

Object	Role
StorageClass	Describes a kind of storage (fast SSD, cheap HDD) and how to provision it
PersistentVolumeClaim (PVC)	An application's request: "I need 10Gi of this class"
PersistentVolume (PV)	The actual disk. Usually created automatically when a claim appears

With dynamic provisioning you only write the claim; the StorageClass creates the disk and the PV for you.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: shop-uploads
  namespace: shop
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: standard
  resources:
    requests:
      storage: 10Gi

# In the Pod template
      containers:
        - name: api
          volumeMounts:
            - name: uploads
              mountPath: /var/lib/shop/uploads
      volumes:
        - name: uploads
          persistentVolumeClaim:
            claimName: shop-uploads
```

Access modes and reclaim policy

Access mode	Meaning
`ReadWriteOnce` (RWO)	Mounted read-write by one node at a time
`ReadOnlyMany` (ROX)	Mounted read-only by many nodes
`ReadWriteMany` (RWX)	Mounted read-write by many nodes; needs a storage system that supports it, such as NFS or a managed file service
`ReadWriteOncePod`	Mounted read-write by a single Pod

The **reclaim policy** decides what happens to the disk when the claim is deleted. `Delete` removes the disk and its data; `Retain` keeps it for manual recovery. Production databases should use a class with `Retain`, or

MARGA by EknathaLabs marga.eknathalabs.com

you should take snapshots, because deleting a claim is one command.

StatefulSets

A Deployment treats Pods as interchangeable. A **StatefulSet** gives each Pod a stable identity (`db-0`, `db-1`, `db-2`), starts and stops them in order, and creates a separate PVC for each through `volumeClaimTemplates`. Use it for databases and clustered systems where each member has its own data and name. A **headless Service** (`clusterIP: None`) gives each Pod its own DNS entry such as `db-0.db.shop.svc.cluster.local`.

```
$ kubectl get pvc -n shop
NAME            STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STOR \
    AGECLASS
data-db-0       Bound    pvc-2f1c8d0e-...                           20Gi       RWO            stan \
    dard
data-db-1       Bound    pvc-7a93b1c4-...                           20Gi       RWO            stan \
    dard
$ kubectl get sc
```

If you only need a single database for a small service, a managed database service from your cloud provider is usually safer than operating one yourself on Kubernetes.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Storage and StatefulSets fits together

Part	What it does	What goes wrong
The three objects	With dynamic provisioning you only write the claim; the StorageClass creates the disk and the PV for you.	Assuming a PVC is deleted with its Pod (it is not) or that data survives claim deletion (it may not).
Access modes and reclaim policy	The reclaim policy decides what happens to the disk when the claim is deleted.	Using ReadWriteOnce volumes for a multi-replica Deployment on different nodes.
StatefulSets	A Deployment treats Pods as interchangeable.	Running databases on emptyDir, which dies with the Pod.

Flow: The three objects to Access modes and reclaim policy / Access modes and reclaim policy to StatefulSets

Incident: the Pod is stuck Pending with an unbound claim

Incident: *A new Pod stays* `Pending` *for ten minutes after you added a volume.*

```
$ kubectl get pvc shop-uploads -n shop
NAME           STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
shop-uploads   Pending                                      fast-ssd
$ kubectl describe pvc shop-uploads -n shop | tail -4
Events:
  Warning  ProvisioningFailed  persistentvolume-controller  storageclass.storage.k8s.io "fast-ssd" \
     not found
$ kubectl get sc
NAME                 PROVISIONER             RECLAIMPOLICY
standard (default)   ebs.csi.aws.com         Delete
```

The claim names a StorageClass that does not exist, so nothing provisions a disk and the Pod cannot be scheduled. Correct `storageClassName` to a real class (here `standard`), delete the pending claim and apply it again, since most claim fields cannot be changed after creation. Other frequent causes: the requested access mode is not supported by the class, the zone has no capacity, or the volume exists in a different availability zone than the node the Pod landed on. Always read the events at the bottom of `kubectl describe`.

Common mistakes

- Assuming a PVC is deleted with its Pod (it is not) or that data survives claim deletion (it may not)
- Using ReadWriteOnce volumes for a multi-replica Deployment on different nodes

- Running databases on `emptyDir`, which dies with the Pod
- Skipping backups because "the disk is persistent"

Check yourself

What is a PersistentVolumeClaim?

Answer: A request for storage made by an application

Which controller gives each Pod a stable name and its own volume?

Answer: StatefulSet

A PVC stays Pending with ProvisioningFailed: storageclass not found. What is the fix?

Answer: Reference a StorageClass that exists

What does the Delete reclaim policy do when the claim is removed?

Answer: Deletes the underlying disk and its data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Jobs, CronJobs, DaemonSets and StatefulSet patterns

15 minutes. Choose the right workload controller for batch work, scheduled tasks and per-node agents, and configure retries, concurrency, history and cleanup.

Not every workload is a web server that runs forever. Database migrations run once, reports run nightly, and log collectors must run on every node. Kubernetes has a controller for each pattern, and picking the wrong one produces duplicated work, missed runs or Pods that never finish.

What you will learn

- Jobs for run-to-completion work, with retries and parallelism
- CronJobs for scheduled work, and their concurrency and history settings
- DaemonSets for one Pod per node
- How to pick between Deployment, StatefulSet, Job and DaemonSet

Jobs

A **Job** creates Pods and keeps retrying until a set number complete successfully. The Pod's restart policy must be `Never` or `OnFailure`.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  namespace: shop
spec:
  backoffLimit: 3
  activeDeadlineSeconds: 600
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: registry.example.com/shop/api:1.8.2
          command: ["python", "manage.py", "migrate"]
          envFrom:
            - secretRef:
                name: shop-db
```

`backoffLimit` caps retries before the Job is marked failed. `activeDeadlineSeconds` kills a Job that runs too long. `ttlSecondsAfterFinished` deletes the finished Job and its Pods automatically, so completed objects do not accumulate. For parallel work, set `completions` and `parallelism`, for example 10 completions with 3 running at a time.

```
$ kubectl apply -f job.yaml
$ kubectl get jobs -n shop
$ kubectl logs job/db-migrate -n shop
$ kubectl describe job db-migrate -n shop | tail -8
$ kubectl wait --for=condition=complete job/db-migrate -n shop --timeout=300s
```

CronJobs

A **CronJob** creates a Job on a schedule, using the same five-field syntax as cron.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
  namespace: shop
spec:
  schedule: "30 2 * * *"
  timeZone: "Etc/UTC"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 600
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 5
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: registry.example.com/shop/report:2.4.0
              args: ["--date", "yesterday"]
```

`concurrencyPolicy` decides what happens if the previous run is still going: `Allow` runs them together, `Forbid` skips the new one, and `Replace` kills the old one. Use `Forbid` for most jobs. `startingDeadlineSeconds` lets a missed run start late within the window. History limits keep recent Jobs around so you can read their logs.

```
$ kubectl get cronjob -n shop
$ kubectl create job manual-report --from=cronjob/nightly-report -n shop
$ kubectl patch cronjob nightly-report -n shop -p '{"spec":{"suspend":true}}'
```

DaemonSets

A **DaemonSet** runs one Pod on every node, or on every node that matches a selector. New nodes get the Pod automatically, and removed nodes lose it. Typical uses are log shippers, metric exporters, storage drivers and network plugins.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      tolerations:
        - operator: Exists
      containers:
        - name: exporter
          image: prom/node-exporter:v1.8.2
          ports:
            - containerPort: 9100
              hostPort: 9100
```

Because control-plane nodes and special nodes carry taints, a DaemonSet needs `tolerations` to run there. Use resource requests so an agent cannot starve the workloads it monitors.

Picking the right controller

Need	Use
Stateless service, interchangeable replicas	Deployment
Stable identity and storage per replica	StatefulSet
Run to completion once	Job
Run on a schedule	CronJob
One copy per node	DaemonSet

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Jobs, CronJobs, DaemonSets and StatefulSet... fits together

Part	What it does	What goes wrong
Jobs	A Job creates Pods and keeps retrying until a set number complete successfully.	Using restartPolicy: Always in a Job template, which is invalid.
CronJobs	A CronJob creates a Job on a schedule, using the same five-field syntax as cron.	Assuming CronJob times are in your local timezone.
DaemonSets	A DaemonSet runs one Pod on every node, or on every node that matches a selector.	Forgetting tolerations so a DaemonSet misses tainted nodes.
Picking the right controller	.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Jobs to CronJobs / CronJobs to DaemonSets / DaemonSets to Picking the right controller

Incident: the report that ran twice and doubled the invoices

> **Incident:** *A nightly invoicing CronJob takes longer than usual because of a slow database. At the next scheduled time the controller starts a second Job while the first is still running. Both read the same unprocessed invoices and customers are billed twice.*

The default `concurrencyPolicy: Allow` permits overlap, and the job was not idempotent. Inspect and fix:

```
$ kubectl get jobs -n billing --sort-by=.metadata.creationTimestamp | tail -3
$ kubectl get cronjob invoicing -n billing -o jsonpath='{.spec.concurrencyPolicy}'
Allow
$ kubectl patch cronjob invoicing -n billing -p '{"spec":{"concurrencyPolicy":"Forbid"}}'
```

Set `Forbid`, make the work idempotent by marking each invoice as claimed in a transaction, add an `activeDeadlineSeconds`, and alert if a run takes longer than expected.

Common mistakes

- Using `restartPolicy: Always` in a Job template, which is invalid
- Leaving finished Jobs forever instead of setting a TTL
- Allowing overlap on jobs that are not idempotent
- Forgetting tolerations so a DaemonSet misses tainted nodes
- Assuming CronJob times are in your local timezone

Check yourself

Which controller runs one Pod on every node?

Answer: DaemonSet

What does concurrencyPolicy: Forbid do?

Answer: Skips a new run if the previous one is still running

What does ttlSecondsAfterFinished do?

Answer: Deletes a completed Job and its Pods after the given time

Why does a DaemonSet often need tolerations?

Answer: To run on nodes that have taints, such as control-plane nodes

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. Resource requests, limits, QoS and quotas

16 minutes. Size Pods with requests and limits, understand QoS classes and OOM kills, and use LimitRange and ResourceQuota to keep a shared cluster fair.

The scheduler places Pods using what they **request**, and the kernel enforces what they are **limited** to. Misunderstanding the difference causes some of the most common Kubernetes incidents: Pods stuck Pending, CPU throttling, and containers killed with "OOMKilled".

What you will learn

- What requests and limits mean for scheduling and for enforcement
- CPU and memory behave differently when a limit is hit
- QoS classes and eviction order
- Namespace controls: LimitRange and ResourceQuota

Requests versus limits

A **request** is the amount the scheduler reserves for the container. A Pod fits on a node only if the sum of requests fits in the node's allocatable resources. A **limit** is the maximum the container may use at runtime.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
  namespace: shop
spec:
  replicas: 3
  selector:
    matchLabels:
      app: shop-api
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: api
          image: registry.example.com/shop/api:1.8.2
          resources:
            requests:
              cpu: 250m
              memory: 256Mi
            limits:
              memory: 512Mi
```

CPU is measured in cores: `250m` is a quarter of a core. Memory is in bytes: `256Mi` is mebibytes. CPU is **compressible**: if a container tries to use more than its CPU limit, it is throttled and slows down. Memory is **incompressible**: if a container exceeds its memory limit the kernel kills it, and the status shows `OOMKilled` with exit code 137.

Many teams set memory requests equal to memory limits, and set CPU requests but omit CPU limits so containers can use idle CPU without throttling. Whatever you choose, measure first and set requests near typical usage.

Seeing real usage

```
$ kubectl top nodes
$ kubectl top pods -n shop --containers
$ kubectl describe node worker-2 | sed -n '/Allocated resources/,/Events/p'
$ kubectl get pod shop-api-7d9c-abcde -n shop -o jsonpath='{.status.containerStatuses[0].lastState \
    .terminated.reason}'
OOMKilled
```

MARGA by EknathaLabs marga.eknathalabs.com

`kubectl top` needs the metrics server. Compare usage over days in Prometheus, not a single moment, before changing requests.

QoS classes

Kubernetes assigns each Pod a Quality of Service class from its requests and limits, which determines who is evicted first under node memory pressure.

Class	Condition	Evicted
Guaranteed	Every container has requests equal to limits for CPU and memory	Last
Burstable	At least one request or limit set, but not Guaranteed	Middle
BestEffort	No requests or limits at all	First

Production workloads should never be BestEffort. Check a Pod's class with `kubectl get pod NAME -o jsonpath='{.status.qosClass}'`.

LimitRange and ResourceQuota

Without guard rails, one team can consume a whole shared cluster. Two namespace-level objects set boundaries.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: shop
spec:
  limits:
    - type: Container
      default:
        memory: 512Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:
        memory: 2Gi
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: shop-quota
  namespace: shop
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.memory: 32Gi
    pods: "40"
    persistentvolumeclaims: "10"
```

A **LimitRange** supplies defaults for containers that omit them and rejects unreasonable sizes. A **ResourceQuota** caps the total a namespace may request. When a quota covers CPU or memory, every Pod must declare them or it is rejected.

```
$ kubectl describe quota shop-quota -n shop
$ kubectl describe limitrange defaults -n shop
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Resource requests, limits, QoS and quotas fits together

Part	What it does	What goes wrong
Requests versus limits	A request is the amount the scheduler reserves for the container.	Setting memory limits far below real peak usage, causing OOM kills.
Seeing real usage	kubectl top needs the metrics server. Compare usage over days in Prometheus, not a single moment, before changing requests.	Believing kubectl top usage determines scheduling.
QoS classes	Kubernetes assigns each Pod a Quality of Service class from its requests and limits, which determines who is evicted first under node memory pressure.	Leaving workloads BestEffort.
LimitRange and ResourceQuota	Without guard rails, one team can consume a whole shared cluster.	Forgetting that a quota forces every Pod to declare resources.

Flow: Requests versus limits to Seeing real usage / Seeing real usage to QoS classes / QoS classes to LimitRange and ResourceQuota

Incident: Pending Pods and a cluster that looks empty

> **Incident:** *New Pods stay* `Pending` *with* `0/6 nodes are available: 6 Insufficient cpu.` *Yet* `kubectl top nodes` *shows every node at 15 percent CPU use. A finance meeting is waiting on the new reporting service.*

The scheduler counts **requests**, not usage. Teams had set generous CPU requests years ago, and the nodes are fully reserved while mostly idle.

```
$ kubectl describe node worker-1 | sed -n '/Allocated resources/,/Events/p'
  cpu                7800m (97%)    0 (0%)
$ kubectl top pods -A --sort-by=cpu | head -5
```

Right-size by lowering requests to match measured usage, using the Vertical Pod Autoscaler in recommendation mode as a guide. Meanwhile add nodes if the service is urgent. Prevent drift with quotas and a periodic review of requested versus used.

Common mistakes

- Setting memory limits far below real peak usage, causing OOM kills
- Setting huge CPU requests "to be safe", wasting capacity
- Leaving workloads BestEffort
- Believing `kubectl top` usage determines scheduling
- Forgetting that a quota forces every Pod to declare resources

Check yourself

What does the scheduler use to decide whether a Pod fits on a node?

Answer: The sum of container resource requests

What happens when a container exceeds its memory limit?

Answer: It is OOM-killed and restarted

Which QoS class is evicted first under memory pressure?

Answer: BestEffort

What does a ResourceQuota do?

Answer: Caps the total resources and object counts a namespace may use

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Autoscaling: HPA, VPA, KEDA and the cluster autoscaler

15 minutes. Scale Pods on CPU, memory and custom metrics with HPA, right-size with VPA, scale to zero with event-driven autoscaling, and add nodes when Pods cannot be scheduled.

Traffic is never flat. Autoscaling adds capacity when demand rises and removes it when demand falls, so you neither fail under load nor pay for idle machines. Kubernetes has several autoscalers that work at different levels, and they must be configured to cooperate.

What you will learn

- The Horizontal Pod Autoscaler and its formula
- The Vertical Pod Autoscaler and when to use it
- Event-driven scaling and scale-to-zero with KEDA
- The cluster autoscaler or Karpenter for nodes

Horizontal Pod Autoscaler

The **HPA** changes the replica count of a Deployment based on observed metrics. The core rule is:

```
desiredReplicas = ceil(currentReplicas x currentMetric / targetMetric)
```

If three Pods run at 90 percent CPU against a 60 percent target, the HPA wants ceil(3 x 90 / 60) = 5 replicas.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shop-api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shop-api
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 25
          periodSeconds: 60
```

CPU utilisation is measured **relative to the Pod's CPU request**, so the Deployment must set requests or the HPA reports `<unknown>`. The `behavior` block slows scale-down to avoid flapping: here the HPA waits five minutes and removes at most a quarter of the Pods per minute.

```
$ kubectl get hpa -n shop
$ kubectl describe hpa shop-api -n shop | sed -n '/Metrics/,/Events/p'
$ kubectl top pods -n shop
$ kubectl get events -n shop --field-selector involvedObject.kind=HorizontalPodAutoscaler
```

Requires the metrics server for CPU and memory. For request rate or queue depth, use a custom metrics adapter such as the Prometheus adapter. CPU is a poor signal for many applications; requests per second or latency often predicts load better.

MARGA by EknathaLabs
marga.eknathalabs.com

Vertical Pod Autoscaler

The **VPA** adjusts container requests and limits. In `Off` mode it only recommends values, which is a safe way to right-size a workload.

```
$ kubectl get vpa shop-api -n shop -o jsonpath='{.status.recommendation.containerRecommendations[0 \
    ]}'
```

Do not let HPA and VPA both act on CPU for the same workload, since they fight. A common pattern is VPA in recommendation mode for sizing and HPA for scaling out.

Event-driven scaling with KEDA

KEDA scales workloads from external event sources such as queue length, Kafka lag or a cron schedule, and can scale to **zero**.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: invoice-worker
  namespace: billing
spec:
  scaleTargetRef:
    name: invoice-worker
  minReplicaCount: 0
  maxReplicaCount: 30
  triggers:
    - type: aws-sqs-queue
      metadata:
        queueURL: https://sqs.eu-west-1.amazonaws.com/123456789012/invoices
        queueLength: "20"
        awsRegion: eu-west-1
```

One worker per twenty queued messages, and none when the queue is empty. Scale-to-zero suits batch consumers; avoid it for services where cold-start latency would hurt users.

Scaling nodes

Pod autoscaling only helps if nodes exist to hold the new Pods. The **cluster autoscaler** (or **Karpenter** on AWS) watches for Pods that are `Pending` because nothing fits and adds nodes, then removes underused nodes.

```
$ kubectl get pods -A --field-selector=status.phase=Pending
$ kubectl get nodes -L node.kubernetes.io/instance-type
$ kubectl -n kube-system logs deploy/cluster-autoscaler | tail -5
```

New nodes take minutes to start, so keep some headroom and use PodDisruptionBudgets so scale-down does not remove too many replicas of one service at once.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Autoscaling: HPA, VPA, KEDA and the cluster... fits together

Part	What it does	What goes wrong
Horizontal Pod Autoscaler	The HPA changes the replica count of a Deployment based on observed metrics.	No scale-down stabilisation, causing flapping.
Vertical Pod Autoscaler	The VPA adjusts container requests and limits.	HPA and VPA both controlling CPU on the same workload.
Event-driven scaling with KEDA	KEDA scales workloads from external event sources such as queue length, Kafka lag or a cron schedule, and can scale to zero.	Using the HPA with no CPU request set.
Scaling nodes	Pod autoscaling only helps if nodes exist to hold the new Pods.	A maximum replica count that exceeds node capacity or quota.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: Horizontal Pod Autoscaler to Vertical Pod Autoscaler / Vertical Pod Autoscaler to Event-driven scaling with KEDA / Event-driven scaling with KEDA to Scaling nodes

Incident: the HPA that could not scale

> **Incident:** *A marketing email triples traffic. Latency climbs, but the HPA stays at 3 replicas.* `kubectl get hpa` *shows* `cpu: <unknown>/60%` *and the team sees no scaling events.*

The HPA had no metric to act on. Find out why:

```
$ kubectl describe hpa shop-api -n shop | tail -8
Warning  FailedGetResourceMetric  missing request for cpu
$ kubectl get deploy shop-api -n shop -o jsonpath='{.spec.template.spec.containers[0].resources}'
{}
```

The Deployment had no CPU request, so utilisation could not be computed. Set a request, confirm `kubectl get hpa` shows a number, load-test before the next campaign, and alert when an HPA reports unknown metrics or sits at its maximum.

Common mistakes

- Using the HPA with no CPU request set
- HPA and VPA both controlling CPU on the same workload
- A maximum replica count that exceeds node capacity or quota
- Scaling on CPU for an I/O-bound application
- No scale-down stabilisation, causing flapping

Check yourself

How does the HPA compute CPU utilisation?

Answer: Relative to the container's CPU request

What does the cluster autoscaler do?

Answer: Adds or removes nodes based on unschedulable Pods and underused nodes

When is KEDA a good fit?

Answer: For workloads driven by events such as queue length, including scale to zero

Why avoid HPA and VPA both managing CPU for one workload?

Answer: They react to each other's changes and can fight

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. RBAC and workload security

17 minutes. Grant least-privilege access with Roles and ServiceAccounts, test it with kubectl auth can-i, and harden Pods with securityContext.

By default, an administrator can do anything in a cluster and so can any careless script that borrows their credentials. Role-based access control (RBAC) limits what each person and workload may do, and Pod security settings limit what a compromised container can do. These are the two layers that most security reviews and the CKS exam focus on.

What you will learn

- Roles, ClusterRoles and bindings
- ServiceAccounts for workloads
- How to test permissions with `kubectl auth can-i`
- A securityContext that follows least privilege

RBAC building blocks

Object	Scope	Purpose
Role	One namespace	A set of allowed verbs on resources
ClusterRole	Whole cluster	Same, for cluster-wide or non-namespaced resources
RoleBinding	One namespace	Grants a Role (or ClusterRole) to users, groups or ServiceAccounts there
ClusterRoleBinding	Whole cluster	Grants a ClusterRole everywhere

RBAC is **additive**: there are no deny rules. A subject can do something only if some binding allows it.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: deployer
  namespace: shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployment-manager
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "update", "patch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deployer-can-manage-deployments
  namespace: shop
subjects:
  - kind: ServiceAccount
    name: deployer
    namespace: shop
roleRef:
  kind: Role
  name: deployment-manager
  apiGroup: rbac.authorization.k8s.io
```

Test rather than assume:

```
$ kubectl auth can-i update deployments -n shop --as=system:serviceaccount:shop:deployer
yes
$ kubectl auth can-i delete secrets -n shop --as=system:serviceaccount:shop:deployer
```

MARGA by EknathaLabs marga.eknathalabs.com

```
no
$ kubectl auth can-i --list -n shop --as=system:serviceaccount:shop:deployer
```

> **Warning:** *Avoid granting* `cluster-admin`*, wildcard verbs (*), or the ability to read Secrets or create Pods broadly. The right to create Pods can usually be turned into the right to read any Secret in the namespace.*

ServiceAccounts for Pods

Every Pod runs as a ServiceAccount, and its token lets it call the API. Most applications never need to call the API, so turn the token off:

```
spec:
  serviceAccountName: shop-api
  automountServiceAccountToken: false
```

securityContext

```
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: api
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
```

These mirror the container hardening you met in the Docker module. **Pod Security Admission** can enforce them per namespace by labelling it, for example `pod-security.kubernetes.io/enforce=restricted`, so non-compliant Pods are rejected at creation.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How RBAC and workload security fits together

Part	What it does	What goes wrong
RBAC building blocks	RBAC is additive: there are no deny rules.	Believing a Role denies things; RBAC only allows.
ServiceAccounts for Pods	Every Pod runs as a ServiceAccount, and its token lets it call the API.	Leaving the default ServiceAccount token mounted in every Pod.
securityContext	These mirror the container hardening you met in the Docker module.	Running containers as root with a writable filesystem.

Flow: RBAC building blocks to ServiceAccounts for Pods / ServiceAccounts for Pods to securityContext

Incident: a CI job fails with "forbidden"

> **Incident:** *The deploy job reports* `deployments.apps "shop-api" is forbidden: User "system:serviceaccount:shop:deployer" cannot patch resource "deployments" in API group "apps" in the namespace "shop"`.

The message names the user, the verb, the resource and the namespace. Check the effective permissions:

```
$ kubectl auth can-i patch deployments -n shop --as=system:serviceaccount:shop:deployer
no
$ kubectl get rolebinding -n shop -o wide
$ kubectl describe role deployment-manager -n shop
```

Typical causes are a missing verb in the Role, a binding in a different namespace, a typo in the ServiceAccount name, or a Role bound to the wrong subject. Add only the missing verb (`patch` here), apply, and re-run `auth can-i`. Resist the urge to bind `cluster-admin` "to make it work". The error message

gave you exactly what the account was missing.

Common mistakes

- Granting cluster-admin to pipelines and humans for convenience
- Leaving the default ServiceAccount token mounted in every Pod
- Running containers as root with a writable filesystem
- Believing a Role denies things; RBAC only allows

Check yourself

Which pair grants a ServiceAccount permissions inside one namespace?

Answer: Role and RoleBinding

Can RBAC explicitly deny an action?

Answer: No, RBAC is purely additive; anything not allowed is denied

Which command tests whether a ServiceAccount can delete Secrets?

Answer: kubectl auth can-i delete secrets -n shop --as=system:serviceaccount:shop:deployer

Which securityContext setting prevents a process from gaining more privileges than its parent?

Answer: allowPrivilegeEscalation: false

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Scheduling and troubleshooting

18 minutes. Control where Pods run with selectors, taints and affinity, and work through the common failure states in a repeatable order.

Two skills separate someone who uses Kubernetes from someone who can run it: placing workloads on purpose, and diagnosing a broken Pod quickly without guessing. This lesson gives you a method for the second, and the tools for the first.

What you will learn

- nodeSelector, affinity, taints and tolerations
- Spreading replicas for availability
- A fixed diagnostic order for failing Pods
- What each common Pod status means

Placement

Tool	Purpose
`nodeSelector`	Run only on nodes with a given label
Node affinity	Same, with richer rules and "preferred" versus "required"
Taint (on a node)	Repel Pods unless they tolerate it
Toleration (on a Pod)	Allows scheduling onto a tainted node
Pod anti-affinity / topology spread	Keep replicas on different nodes or zones

Taints and tolerations are for *keeping others away*: dedicate GPU nodes by tainting them, and only GPU workloads tolerate the taint. Affinity is for *attracting* Pods.

```
$ kubectl label node worker-3 disktype=ssd
$ kubectl taint nodes worker-3 dedicated=gpu:NoSchedule
```

Spread replicas so one node or zone failure does not take all of them:

```
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: shop-api
```

Protect availability during voluntary disruptions such as node drains with a **PodDisruptionBudget**:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shop-api
  namespace: shop
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: shop-api
```

The diagnostic order

Do the same steps every time, from broad to specific:

```
$ kubectl get pods -n shop                      # status and restarts
$ kubectl describe pod NAME -n shop             # events at the bottom
$ kubectl logs NAME -n shop --previous          # logs of the crashed container
$ kubectl get events -n shop --sort-by=.lastTimestamp
$ kubectl get endpoints,svc -n shop             # networking
$ kubectl top pod -n shop                       # resource use, if metrics are installed
```

MARGA by EknathaLabs marga.eknathalabs.com

Status	Meaning	Usual cause and first step
`Pending`	Not scheduled yet	No node fits (requests too big, taints, unbound PVC). `describe` shows `FailedScheduling`
`ImagePullBackOff` / `ErrImagePull`	Cannot fetch the image	Wrong name or tag, or missing registry credentials
`CrashLoopBackOff`	Container keeps exiting	Application error or bad config. Read `logs --previous`
`OOMKilled` (in last state)	Exceeded memory limit	Raise the limit or fix the leak
`CreateContainerConfigError`	Missing ConfigMap, Secret or key	Create it or correct the reference
`Running` but `0/1` ready	Readiness probe failing	Check probe path, port, app start-up
`Terminating` for long	Finalizer or stuck process	Investigate the cause before forcing deletion

For a node that is unhealthy, `kubectl describe node` shows conditions such as `MemoryPressure` and `DiskPressure`. To empty a node for maintenance without downtime, use `kubectl drain worker-2 --ignore-daemonsets --delete-emptydir-data`, which respects PodDisruptionBudgets, then `kubectl uncordon worker-2` when finished.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Scheduling and troubleshooting fits together

Part	What it does	What goes wrong
Placement	Taints and tolerations are for keeping others away: dedicate GPU nodes by tainting them, and only GPU workloads tolerate the taint.	Draining nodes with no PodDisruptionBudget on critical services.
The diagnostic order	Do the same steps every time, from broad to specific.	Forcing deletion of Pods without understanding why they are stuck.
Incident: a Pod will not schedule	Read the message literally: of four nodes, one is tainted and the Pod has no toleration, and the other three do not have enough free memory for the request (the...	Reading kubectl logs without --previous after a crash.

Flow: Placement to The diagnostic order / The diagnostic order to Incident: a Pod will not schedule

Incident: a Pod will not schedule

Incident: *The new* `batch` *Deployment shows 0 of 3 Pods running. All three are* `Pending`*.*

```
$ kubectl describe pod batch-5f9c7b8d6-abcde -n shop | tail -4
Events:
  Warning  FailedScheduling  default-scheduler  0/4 nodes are available: 1 node(s) had untolerated \
     taint {dedicated: gpu}, 3 Insufficient memory.
```

Read the message literally: of four nodes, one is tainted and the Pod has no toleration, and the other three do not have enough free memory for the **request** (the scheduler places by requests, not by current use). Check what is being asked and what is free:

```
$ kubectl get deploy batch -n shop -o jsonpath='{.spec.template.spec.containers[0].resources}'
$ kubectl describe node worker-1 | grep -A6 "Allocated resources"
```

If the request of 8Gi is far larger than the workload needs, lower it. If it is genuinely needed, add capacity or let a cluster autoscaler add a node. Do not add a toleration for the GPU taint just to get scheduled, since that defeats the reason the taint exists.

Common mistakes

- Reading `kubectl logs` without `--previous` after a crash
- Skipping `describe` and its events
- Putting every replica on one node
- Draining nodes with no PodDisruptionBudget on critical services
- Forcing deletion of Pods without understanding why they are stuck

Check yourself

A Pod is CrashLoopBackOff. Which command shows why the previous container exited?

Answer: kubectl logs POD --previous

What do taints and tolerations do?

Answer: A taint repels Pods from a node unless they have a matching toleration

FailedScheduling says Insufficient memory but the nodes look mostly idle. Why?

Answer: The scheduler places Pods by their resource requests, not current usage

What does a PodDisruptionBudget protect against?

Answer: Too many Pods being voluntarily evicted at once, such as during a drain

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Helm and Kustomize

16 minutes. Package and configure Kubernetes applications with Helm charts and values, overlay environments with Kustomize, and release, upgrade and roll back safely.

Raw YAML does not scale. The same Deployment needs different replicas, hostnames and secrets in development, staging and production, and copying files invites drift. Two tools solve this in different ways: **Helm** templates and packages applications, and **Kustomize** patches plain YAML without templating.

What you will learn

- Helm concepts: chart, values, release, repository
- Installing, upgrading and rolling back a release
- Reading and writing chart templates
- Kustomize bases and overlays, and when to choose which

Helm concepts

A **chart** is a package of templates plus default values. A **release** is one installed instance of a chart in a cluster, with a name and a revision history. **Values** are the settings that fill the templates.

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm repo update
$ helm search repo nginx --versions | head -4
$ helm show values bitnami/nginx | head -30
$ helm install web bitnami/nginx -n shop --create-namespace -f values-prod.yaml
$ helm list -n shop
$ helm status web -n shop
```

Always keep your overrides in a values file stored in version control, not typed as `--set` flags, so the deployment is reproducible and reviewable.

```
replicaCount: 3
image:
  tag: "1.27.1"
resources:
  requests:
    cpu: 100m
    memory: 128Mi
ingress:
  enabled: true
  hostname: shop.example.com
```

Upgrade, diff and rollback

```
$ helm template web bitnami/nginx -f values-prod.yaml | less
$ helm upgrade web bitnami/nginx -n shop -f values-prod.yaml --atomic --timeout 5m
$ helm history web -n shop
$ helm rollback web 2 -n shop
$ helm get values web -n shop
$ helm uninstall web -n shop
```

`helm template` renders the final YAML without touching the cluster, which is the quickest way to see what a change will do. `--atomic` rolls back automatically if the upgrade fails. The `helm diff` plugin shows the changes against the live release before you apply them. Lint charts you write with `helm lint`.

Writing a chart

`helm create shop` scaffolds a chart. The key files are `Chart.yaml` (name and version), `values.yaml` (defaults) and `templates/` (Go templates).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "shop.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    matchLabels:
      app: {{ include "shop.name" . }}
  template:
    metadata:
      labels:
        app: {{ include "shop.name" . }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          {{- with .Values.resources }}
          resources:
            {{- toYaml . | nindent 12 }}
          {{- end }}
```

`{{ .Values.x }}` reads a value, `include` reuses a helper, and `toYaml | nindent` inserts a block with correct indentation. Indentation mistakes are the most common template bug, so render with `helm template` and validate the result.

Kustomize

Kustomize starts from plain, valid YAML (the **base**) and applies **overlays** that patch it. It is built into `kubectl`.

```
app/
  base/
    deployment.yaml
    service.yaml
    kustomization.yaml
  overlays/
    staging/
      kustomization.yaml
    production/
      kustomization.yaml
      replica-patch.yaml
```

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namespace: shop
images:
  - name: registry.example.com/shop/api
    newTag: 1.8.2
patches:
  - path: replica-patch.yaml
commonLabels:
  env: production
```

```
$ kubectl kustomize overlays/production | head -30
$ kubectl apply -k overlays/production
$ kubectl diff -k overlays/production
```

Choosing between them

Situation	Choice
Installing third-party software (databases, ingress controllers)	Helm, using the vendor chart
Your own app with a few environment differences	Kustomize overlays
Needs conditionals, loops and versioned packages	Helm
You want the YAML to stay readable and template-free	Kustomize

They combine well: render a Helm chart and patch the output with Kustomize, or let Argo CD deploy either. Whatever you pick, deploy through a pipeline, never by hand from a laptop.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Helm and Kustomize fits together

Part	What it does	What goes wrong
Helm concepts	A chart is a package of templates plus default values.	Using --set for production settings that are not stored anywhere.
Upgrade, diff and rollback	helm template renders the final YAML without touching the cluster, which is the quickest way to see what a change will do.	Upgrading charts without reading the changelog and diff.
Writing a chart	helm create shop scaffolds a chart. The key files are Chart.yaml (name and version), values.yaml (defaults) and templates/ (Go templates).	Template indentation errors that produce invalid YAML.
Kustomize	Kustomize starts from plain, valid YAML (the base) and applies overlays that patch it.	Putting secrets in values files in plain text.
Choosing between them	They combine well: render a Helm chart and patch the output with Kustomize, or let Argo CD deploy either.	Using both Helm and manual kubectl edit on the same objects.

Flow: Helm concepts to Upgrade, diff and rollback / Upgrade, diff and rollback to Writing a chart / Writing a chart to Kustomize / Kustomize to Choosing between them

Incident: the upgrade that wiped the customisations

> **Incident:** *An engineer upgrades a release with* `helm upgrade web bitnami/nginx --version 18.2.0` *and no values file. The site reverts to one replica and the default hostname, because the previous overrides were passed with* `--set` *and were not retained.*

Helm builds the new release from the chart defaults plus whatever you pass this time. Recover and prevent:

```
$ helm history web -n shop
$ helm rollback web 4 -n shop
$ helm get values web -n shop -o yaml > recovered-values.yaml
$ helm upgrade web bitnami/nginx -n shop -f recovered-values.yaml --version 18.2.0 --atomic
```

Keep all values in files under version control, use the same files in every upgrade, preview with `helm template` or `helm diff`, and let a pipeline run the command so the inputs are always the same.

Common mistakes

- Using `--set` for production settings that are not stored anywhere
- Upgrading charts without reading the changelog and diff
- Template indentation errors that produce invalid YAML
- Putting secrets in values files in plain text
- Using both Helm and manual `kubectl edit` on the same objects

Check yourself

What is a Helm release?

Answer: An installed instance of a chart in a cluster, with a revision history

Which command renders Helm templates without changing the cluster?

Answer: helm template

How does Kustomize differ from Helm?

Answer: It patches plain YAML with overlays instead of using templates

Why use helm upgrade --atomic?

Answer: To roll back automatically if the upgrade fails

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Custom resources, controllers and operators

15 minutes. Extend the Kubernetes API with CustomResourceDefinitions, understand the controller reconcile loop, and run production software with operators.

Kubernetes ships with Deployments, Services and Pods, but its API is extensible. Databases, certificate managers, service meshes and delivery tools all add their own resource kinds. Understanding this model explains how most of the cloud-native ecosystem works.

What you will learn

- What a CustomResourceDefinition (CRD) is
- The reconcile loop that powers every controller
- How operators automate running complex software
- Inspecting and troubleshooting custom resources

CustomResourceDefinitions

A **CRD** registers a new resource type with the API server. After that, you can create, list, watch and delete objects of that type with `kubectl`, and they get RBAC, validation and storage like built-in kinds.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.ops.example.com
spec:
  group: ops.example.com
  scope: Namespaced
  names:
    plural: backups
    singular: backup
    kind: Backup
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["schedule", "target"]
              properties:
                schedule:
                  type: string
                target:
                  type: string
                retentionDays:
                  type: integer
                  minimum: 1
```

The schema gives validation: a Backup without a `schedule` is rejected at creation. A custom resource is only data until something acts on it.

```
apiVersion: ops.example.com/v1
kind: Backup
metadata:
  name: orders-db
  namespace: shop
spec:
  schedule: "0 2 * * *"
  target: orders-db
  retentionDays: 14
```

MARGA by EknathaLabs marga.eknathalabs.com

The controller pattern

A **controller** is a program that runs a loop: observe the actual state, compare it with the desired state in the object's `spec`, and act to close the gap. This is **reconciliation**, and it is level-based: the controller does not need to know what changed, only what is wanted now.

```
watch Backup objects
for each Backup:
    desired = spec (schedule, target, retention)
    actual  = existing CronJob, storage, last status
    if actual != desired: create or patch resources
    write result to status
```

Built-in controllers work the same way. The Deployment controller sees `replicas: 3`, finds two Pods and creates a third. Because the loop repeats continuously, the system heals itself after failures. Controllers report progress in `status` and `conditions`, which is where you look first when something is wrong.

Operators

An **operator** is a controller plus a CRD that encodes human operational knowledge for a specific application: how to install it, back it up, scale it, upgrade it and recover it. A PostgreSQL operator might turn this into a running, replicated, backed-up cluster:

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: orders-db
  namespace: shop
spec:
  instances: 3
  storage:
    size: 50Gi
  backup:
    retentionPolicy: "14d"
```

Operators exist for databases, message queues, certificate management (cert-manager), monitoring (Prometheus Operator) and many other systems. They are the usual way to run stateful software on Kubernetes, because they capture failure handling you would otherwise do by hand.

Working with custom resources

```
$ kubectl get crd | head
$ kubectl get crd backups.ops.example.com -o jsonpath='{.spec.versions[*].name}'
$ kubectl api-resources --api-group=ops.example.com
$ kubectl explain backup.spec
$ kubectl get backups -n shop
$ kubectl describe backup orders-db -n shop
$ kubectl get backup orders-db -n shop -o jsonpath='{.status.conditions}'
```

`kubectl explain` documents the schema of any resource, including custom ones. When an object misbehaves, read its `status`, then the events, then the **controller's logs**.

```
$ kubectl get pods -A | grep -i operator
$ kubectl logs deploy/backup-operator -n ops-system --tail=50
$ kubectl get events -n shop --sort-by=.lastTimestamp | tail
```

Running operators responsibly

- Pin operator and CRD versions and test upgrades in staging
- Upgrade CRDs before the operator that uses them
- Deleting a CRD **deletes every object of that kind**, and sometimes their managed resources
- Give operators only the RBAC they need; they are powerful controllers
- Back up the data operators manage, not just their YAML

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Custom resources, controllers and operators fits together

Part	What it does	What goes wrong
CustomResourceDefinitions	A CRD registers a new resource type with the API server.	Debugging only the custom resource and never the operator's logs.
The controller pattern	A controller is a program that runs a loop: observe the actual state, compare it with the desired state in the object's spec, and act to close the gap.	Treating a custom resource as if it had no controller behind it.
Operators	An operator is a controller plus a CRD that encodes human operational knowledge for a specific application: how to install it, back it up, scale it, upgrade it and...	Installing operators from unreviewed manifests with cluster-admin rights.
Working with custom resources	kubectl explain documents the schema of any resource, including custom ones.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Running operators responsibly	Pin operator and CRD versions and test upgrades in staging.	Deleting CRDs without understanding the cascade.

Flow: CustomResourceDefinitions to The controller pattern / The controller pattern to Operators / Operators to Working with custom resources / Working with custom resources to Running operators responsibly

Incident: the CRD that took the database with it

> **Incident:** *During a clean-up an engineer deletes the PostgreSQL operator's CRDs with* `kubectl delete crd` *to "reinstall them fresh". Every database custom resource disappears, and the operator, seeing them removed, begins deleting the managed clusters.*

Deleting a CRD cascades to all its objects. Stop the damage and recover:

```
$ kubectl scale deploy/cnpg-controller-manager -n cnpg-system --replicas=0
$ kubectl get pvc -n shop
$ kubectl get volumesnapshot -n shop
```

Stop the operator first so it cannot delete more, check whether the PersistentVolumes survived (reclaim policy `Retain` saves them), and restore from backup or snapshots. Afterwards protect against it: use RBAC so only a few people can delete CRDs, set PV reclaim policy to `Retain` for databases, take backups outside the cluster and rehearse the restore.

Common mistakes

- Deleting CRDs without understanding the cascade
- Treating a custom resource as if it had no controller behind it
- Installing operators from unreviewed manifests with cluster-admin rights
- Skipping CRD upgrades documented in the release notes
- Debugging only the custom resource and never the operator's logs

Check yourself

What does a CustomResourceDefinition do?

Answer: Registers a new resource type with the Kubernetes API

What is the reconcile loop?

Answer: A controller repeatedly compares desired and actual state and corrects differences

What is an operator?

Answer: A controller and CRD that automate running a specific application

MARGA by EknathaLabs marga.eknathalabs.com

What happens when you delete a CRD?

Answer: All custom resources of that kind are deleted too

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. CNI, Pod networking and network policies

16 minutes. Understand how Pods get IP addresses and reach each other across nodes, what a CNI plugin does, how kube-proxy implements Services, and how to isolate traffic with NetworkPolicy.

Every Pod gets its own IP address and can talk to every other Pod without NAT. That promise sounds simple, but a plugin has to deliver it on real networks. Knowing how makes the difference between guessing and diagnosing when connections fail.

What you will learn

- The Kubernetes network model and its three rules
- What a CNI plugin does and how Calico, Cilium and Flannel differ
- How a Service becomes iptables or IPVS rules via kube-proxy
- Default-deny NetworkPolicy and how to test it

The network model

1. Every Pod gets a unique, cluster-wide IP address
2. Pods can communicate with all other Pods without NAT
3. Agents on a node can communicate with all Pods on that node

Service addresses are **virtual**: no interface owns a ClusterIP. Traffic to it is rewritten to a healthy backend Pod's IP.

```
$ kubectl get pods -n shop -o wide
$ kubectl get svc shop-api -n shop
$ kubectl get endpointslices -n shop -l kubernetes.io/service-name=shop-api
$ kubectl cluster-info dump | grep -m1 -E "cluster-cidr|service-cluster-ip-range"
```

The CNI plugin

The **Container Network Interface (CNI)** is a standard that lets the kubelet ask a plugin to give a new Pod a network interface and an address. Popular plugins differ in how they carry traffic between nodes:

Plugin	Approach	Notable features
Flannel	Simple VXLAN overlay	Minimal, no NetworkPolicy
Calico	Routed (BGP) or overlay	Mature NetworkPolicy, scalable
Cilium	eBPF dataplane	Fast, rich policy, observability, can replace kube-proxy
Cloud CNIs (AWS VPC CNI, Azure CNI)	Pods get real VPC or VNet addresses	Native routing, but consume subnet addresses

An **overlay** wraps Pod traffic in another packet (VXLAN or similar) to cross nodes, which works anywhere but adds overhead. **Routed** designs advertise Pod CIDRs so the underlying network delivers them directly. Choose at cluster creation: switching plugins later is disruptive.

```
$ ls /etc/cni/net.d/
$ kubectl get pods -n kube-system -o wide | grep -E "calico|cilium|flannel|aws-node"
$ kubectl get nodes -o custom-columns=NAME:.metadata.name,PODCIDR:.spec.podCIDR
$ ip route | grep -E "cali|cilium|vxlan|flannel"
```

If new Pods hang in `ContainerCreating` with "network plugin not ready" or "failed to set up sandbox", the CNI plugin on that node is not healthy. Check its DaemonSet Pod.

How Services work

kube-proxy runs on every node, watches Services and EndpointSlices, and programs the kernel (iptables or IPVS rules, or nothing at all when Cilium replaces it) to rewrite ClusterIP traffic to a backend.

MARGA by EknathaLabs marga.eknathalabs.com

```
$ sudo iptables -t nat -L KUBE-SERVICES -n | head -8
$ sudo ipvsadm -Ln | head
$ kubectl get cm kube-proxy -n kube-system -o jsonpath='{.data.config\.conf}' | grep mode
$ kubectl run nettest --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- -T 3 http://shop- \
    api.shop.svc.cluster.local
```

A Service with no endpoints usually means its selector does not match any Pod labels or the Pods are not Ready.

NetworkPolicy

By default every Pod can reach every other Pod. A **NetworkPolicy** restricts that, but only if the CNI enforces it (Flannel alone does not). Start with default deny, then open what is needed.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app: shop-api
  policyTypes: ["Ingress", "Egress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx
      ports:
        - port: 8080
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: orders-db
      ports:
        - port: 5432
    - to:
        - namespaceSelector: {}
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - port: 53
          protocol: UDP
```

Notice the DNS egress rule. After a default-deny on egress, forgetting DNS breaks name resolution for every Pod.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How CNI, Pod networking and network policies fits together

Part	What it does	What goes wrong
The network model	Every Pod gets a unique, cluster-wide IP address.	Pod CIDR that overlaps with VPC or on-premises ranges.
The CNI plugin	The Container Network Interface (CNI) is a standard that lets the kubelet ask a plugin to give a new Pod a network interface and an address.	Running out of subnet addresses with a VPC-native CNI.
How Services work	kube-proxy runs on every node, watches Services and EndpointSlices, and programs the kernel (iptables or IPVS rules, or nothing at all when Cilium replaces it) to...	Confusing a Service with no endpoints for a network problem.
NetworkPolicy	By default every Pod can reach every other Pod.	Default-deny egress without allowing DNS.

Flow: The network model to The CNI plugin / The CNI plugin to How Services work / How Services work to NetworkPolicy

Incident: everything times out after the security hardening

Incident: *A security sprint adds a default-deny NetworkPolicy to the* `shop` *namespace. Minutes later the API reports* `Temporary failure in name resolution` *and checkout stops working.*

Egress to CoreDNS was blocked along with everything else.

```
$ kubectl exec deploy/shop-api -n shop -- nslookup orders-db
;; connection timed out; no servers could be reached
$ kubectl get networkpolicy -n shop
$ kubectl describe networkpolicy default-deny -n shop | tail -6
```

Add an egress rule allowing UDP and TCP port 53 to the DNS Pods, and then rules for each real dependency. Roll out policies in staging first, watch for denied flows with the CNI's observability tools, and keep a tested rollback.

Common mistakes

- Writing NetworkPolicies on a CNI that does not enforce them
- Default-deny egress without allowing DNS
- Pod CIDR that overlaps with VPC or on-premises ranges
- Running out of subnet addresses with a VPC-native CNI
- Confusing a Service with no endpoints for a network problem

Check yourself

What does a CNI plugin do?

Answer: Gives Pods network interfaces and addresses and connects them across nodes

What does kube-proxy do?

Answer: Programs node rules so Service virtual IPs reach backend Pods

What is true about NetworkPolicy by default?

Answer: Without policies all Pods can talk to each other, and enforcement needs a supporting CNI

After applying a default-deny egress policy, what is commonly forgotten?

Answer: An egress rule allowing DNS to CoreDNS

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 14. Building, upgrading and backing up a cluster

18 minutes. Bootstrap a cluster with kubeadm, upgrade it one minor version at a time, back up and restore etcd, and manage certificates and node maintenance.

Managed services hide this work, but exams, on-premises clusters and many incidents depend on it. If you understand how a cluster is built, upgraded and recovered, you can reason about any Kubernetes environment.

What you will learn

- Node prerequisites and a `kubeadm` bootstrap
- The safe upgrade order for control plane and workers
- Backing up and restoring etcd
- Certificate expiry and node drain and maintenance

Preparing nodes

Each node needs a container runtime, the kubelet, kubeadm and kubectl at compatible versions, swap disabled, the right kernel modules and sysctl settings, and unique hostnames and MAC addresses.

```
$ sudo swapoff -a
$ sudo sed -i '/ swap / s/^/#/' /etc/fstab
$ printf 'overlay\nbr_netfilter\n' | sudo tee /etc/modules-load.d/k8s.conf
$ printf 'net.bridge.bridge-nf-call-iptables = 1\nnet.ipv4.ip_forward = 1\n' | sudo tee /etc/sysct \
    l.d/k8s.conf
$ sudo sysctl --system
$ sudo apt-mark hold kubelet kubeadm kubectl
```

Holding the Kubernetes packages stops a routine `apt upgrade` from moving the cluster to a new version unannounced.

Bootstrapping with kubeadm

```
$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --control-plane-endpoint=k8s-api.example.com
$ mkdir -p $HOME/.kube && sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown "$(id -u):$(id -g)" $HOME/.kube/config
$ kubectl get nodes
$ kubectl apply -f cni-manifest.yaml
$ sudo kubeadm token create --print-join-command
$ sudo kubeadm join k8s-api.example.com:6443 --token abcdef.0123456789abcdef --discovery-token-ca- \
    cert-hash sha256:1234
```

`kubeadm init` generates certificates, writes static Pod manifests for the API server, controller manager, scheduler and etcd into `/etc/kubernetes/manifests/`, and starts the kubelet. Nodes stay `NotReady` until a CNI plugin is installed. Use a stable DNS name or load balancer for the control-plane endpoint so you can add more control-plane nodes later.

Upgrading

Kubernetes supports upgrading **one minor version at a time** (for example 1.30 to 1.31, never 1.29 to 1.31). Read the release notes and deprecation list first, check API versions your manifests use, and back up etcd before you start.

Order matters: the control plane first, then the workers, with components never newer than the API server.

```
$ sudo apt-mark unhold kubeadm && sudo apt-get install -y kubeadm=1.31.2-1.1 && sudo apt-mark hold \
    kubeadm
$ sudo kubeadm upgrade plan
$ sudo kubeadm upgrade apply v1.31.2
$ kubectl drain cp-1 --ignore-daemonsets --delete-emptydir-data
$ sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
$ sudo systemctl daemon-reload && sudo systemctl restart kubelet
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ kubectl uncordon cp-1
```

On each worker: upgrade kubeadm, run `sudo kubeadm upgrade node`, drain the node, upgrade the kubelet, restart it and uncordon. Do one node at a time, and respect PodDisruptionBudgets so an application keeps enough replicas during the drain.

Backing up etcd

etcd stores every object in the cluster. Losing it without a backup means losing the cluster's state.

```
$ sudo ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db \
    --endpoints=https://127.0.0.1:2379 \
    --cacert=/etc/kubernetes/pki/etcd/ca.crt \
    --cert=/etc/kubernetes/pki/etcd/server.crt \
    --key=/etc/kubernetes/pki/etcd/server.key
$ sudo ETCDCTL_API=3 etcdutl snapshot status /backup/etcd-2025-06-02.db -w table
```

Store snapshots off the node, encrypt them (they contain Secrets), and schedule them. A backup you have never restored is a hope, not a backup.

```
$ sudo systemctl stop kubelet
$ sudo etcdutl snapshot restore /backup/etcd-2025-06-02.db --data-dir /var/lib/etcd-restore
$ sudo sed -i 's#/var/lib/etcd#/var/lib/etcd-restore#' /etc/kubernetes/manifests/etcd.yaml
$ sudo systemctl start kubelet
$ kubectl get nodes
```

The restore writes a new data directory, and you point the etcd static Pod at it.

Certificates and maintenance

kubeadm certificates expire after a year unless renewed, and upgrades renew them. An expired certificate makes the API server unreachable.

```
$ sudo kubeadm certs check-expiration
$ sudo kubeadm certs renew all
$ kubectl cordon worker-2
$ kubectl drain worker-2 --ignore-daemonsets --delete-emptydir-data --grace-period=60
$ kubectl uncordon worker-2
```

`cordon` stops new Pods landing on a node; `drain` also evicts the existing ones safely; `uncordon` returns it to service.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Building, upgrading and backing up a cluster fits together

Part	What it does	What goes wrong
Preparing nodes	Each node needs a container runtime, the kubelet, kubeadm and kubectl at compatible versions, swap disabled, the right kernel modules and sysctl settings, and unique...	Skipping minor versions during an upgrade.
Bootstrapping with kubeadm	kubeadm init generates certificates, writes static Pod manifests for the API server, controller manager, scheduler and etcd into /etc/kubernetes/manifests/, and starts...	Upgrading workers before the control plane.
Upgrading	Kubernetes supports upgrading one minor version at a time (for example 1.30 to 1.31, never 1.29 to 1.31).	No tested etcd restore procedure.
Backing up etcd	etcd stores every object in the cluster.	Draining without PodDisruptionBudgets for critical services.
Certificates and maintenance	kubeadm certificates expire after a year unless renewed, and upgrades renew them.	Ignoring certificate expiry dates.

Flow: Preparing nodes to Bootstrapping with kubeadm / Bootstrapping with kubeadm to Upgrading / Upgrading to Backing up etcd / Backing up etcd to Certificates and maintenance

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: kubectl stops working one year after install

> **Incident:** *On a Monday morning every* `kubectl` *command fails with* `x509: certificate has expired or is not yet valid`*. The cluster was built exactly a year ago and nobody upgraded it since.*

The control-plane certificates expired. Workloads keep running, but nothing can be managed.

```
$ sudo kubeadm certs check-expiration | head -6
$ sudo kubeadm certs renew all
$ sudo crictl ps | grep -E "kube-apiserver|etcd"
$ sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
$ kubectl get nodes
```

Renew the certificates, restart the static Pods (move their manifests out and back, or restart the kubelet), refresh the admin kubeconfig and verify. Add a calendar alert and a Prometheus rule on certificate expiry, and upgrade the cluster at least yearly.

Common mistakes

- Skipping minor versions during an upgrade
- Upgrading workers before the control plane
- No tested etcd restore procedure
- Ignoring certificate expiry dates
- Draining without PodDisruptionBudgets for critical services

Check yourself

In which order should a kubeadm cluster be upgraded?

Answer: Control plane first, then workers one at a time

How many minor versions can you skip when upgrading?

Answer: None, upgrade one minor version at a time

Why encrypt etcd snapshots?

Answer: They contain every object in the cluster, including Secrets

What does kubectl drain do?

Answer: Cordons the node and safely evicts its Pods

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 15. CKA and CKS exam strategy

14 minutes. Prepare for the hands-on Kubernetes certifications with a speed setup, imperative commands, and a time plan, and see what each exam covers.

The Certified Kubernetes Administrator (CKA) and Certified Kubernetes Security Specialist (CKS) are performance-based exams: you solve tasks at a real terminal against real clusters, under a time limit. Knowing the material is not enough. Speed and accuracy at the command line decide the result.

What you will learn

- What each exam tests
- Terminal setup that saves minutes
- Imperative commands that generate YAML
- A time-management plan

> **Exam tip:** *Exam details change. Domain weights, the Kubernetes version, the allowed documentation and the passing score are published by the Linux Foundation and the CNCF. Check the current candidate handbook before you book, and treat the numbers in this lesson as an orientation only.*

What the exams cover

	CKA	CKS
Focus	Administering a cluster	Securing a cluster and its workloads
Typical areas	Cluster setup and upgrades, workloads and scheduling, services and networking, storage, troubleshooting	Cluster hardening, system hardening, minimizing microservice vulnerabilities, supply chain security, monitoring and runtime security
Prerequisite	None	An active CKA
Format	Hands-on tasks in a browser terminal	Hands-on tasks in a browser terminal

Every lesson in this module maps to CKA areas. The security lesson, plus topics such as audit logs, runtime detection and image scanning, build toward CKS.

Terminal setup

Spend the first minute on a small set of aliases and completion. Many exam environments already provide `k` and completion, so check before typing.

```
alias k=kubectl
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
export do="--dry-run=client -o yaml"
export now="--force --grace-period=0"
```

Configure `vim` for YAML so pasted indentation does not break:

```
cat > ~/.vimrc <<'EOF'
set expandtab
set tabstop=2
set shiftwidth=2
EOF
```

Generate YAML, do not type it

Imperative commands create manifests for you in seconds. Redirect the output to a file, edit the few fields you need, then apply.

```
$ k run web --image=nginx:1.27 $do > pod.yaml
$ k create deployment api --image=nginx:1.27 --replicas=3 $do > deploy.yaml
$ k expose deployment api --port=80 --target-port=8080 $do > svc.yaml
$ k create configmap cfg --from-literal=LOG_LEVEL=debug $do
$ k create role reader --verb=get,list --resource=pods $do
$ k create rolebinding rb --role=reader --serviceaccount=default:app $do
$ k explain pod.spec.securityContext
```

MARGA by EknathaLabs marga.eknathalabs.com

When you forget a field, `k explain --recursive pod.spec | less` is faster than searching the documentation. Learn the layout of the allowed documentation site and use its search for things such as NetworkPolicy and PersistentVolume examples.

Tasks to be able to do from memory

- Upgrade a cluster with the node-by-node procedure (`kubeadm upgrade`, drain, upgrade kubelet, uncordon)
- Back up and restore etcd with `etcdctl snapshot`
- Create and fix RBAC, NetworkPolicies, PVCs and Ingress
- Troubleshoot a node whose kubelet is stopped: `systemctl status kubelet`, `journalctl -u kubelet`
- Find why a Pod is Pending or crashing using the order from the previous lesson
- For CKS: AppArmor or seccomp profiles, Pod Security Admission, image policy, audit policy, and tools such as Trivy and Falco

Time plan

Exams commonly contain 15 to 20 tasks in about two hours, each with a weight. A workable approach:

1. Read the whole task list and note the weights. Do quick, high-weight tasks first.
2. Switch to the correct context at the start of **every** task. The task text tells you the command to run.
3. If a task is not progressing after about six to eight minutes, flag it and move on.
4. Verify each answer: run `get`, `describe`, or a test request before moving on.
5. Keep the last ten minutes for flagged tasks.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How CKA and CKS exam strategy fits together

Part	What it does	What goes wrong
What the exams cover	Every lesson in this module maps to CKA areas.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Terminal setup	Spend the first minute on a small set of aliases and completion.	Typing YAML from scratch instead of generating it.
Generate YAML, do not type it	Imperative commands create manifests for you in seconds.	Generate YAML, do not type it Imperative commands create manifests for you in seconds.
Tasks to be able to do from memory	Upgrade a cluster with the node-by-node procedure (kubeadm upgrade, drain, upgrade kubelet, uncordon).	Check the Common mistakes list below this diagram for what usually goes wrong here.
Time plan	Exams commonly contain 15 to 20 tasks in about two hours, each with a weight.	Not setting the context for each task.

Flow: What the exams cover to Terminal setup / Terminal setup to Generate YAML, do not type it / Generate YAML, do not type it to Tasks to be able to do from memory / Tasks to be able to do from memory to Time plan

Incident: right answer, wrong cluster

> **Incident:** *You spend ten minutes solving a task perfectly, then the check fails. The task said to use context* `k8s-b`*, and you were still on* `k8s-a`*.*

Every task begins with a context command, and skipping it is the most common way to lose points on a task you actually solved. Build the habit: copy the context command from the task, run it, confirm with `k`

`config current-context`, and only then start. This is the same lesson as the first one in this module, except here the cost is marks instead of a production outage.

Practise by timing yourself on a local cluster. Create a deliberately broken setup (a wrong image tag, a missing key, a bad selector), and fix it using only the commands in this module. Do that until the diagnostic order is automatic.

Common mistakes

- Typing YAML from scratch instead of generating it
- Not setting the context for each task
- Spending 20 minutes on one hard task
- Not verifying the result
- Studying only theory and never practising under a clock

Check yourself

What is the fastest way to create a base Deployment manifest in an exam?

Answer: kubectl create deployment ... --dry-run=client -o yaml > file

What must you do at the start of every exam task?

Answer: Switch to the context named in the task and confirm it

A task has stalled for ten minutes with little progress. What is the best move?

Answer: Flag it, move on, and return if time allows

Which exam requires an active CKA as a prerequisite?

Answer: CKS

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. Probes, metrics and logging on Kubernetes

15 minutes. Configure liveness, readiness and startup probes correctly, collect resource and application metrics, and ship logs so a running cluster explains itself.

Kubernetes restarts failed containers and removes unready Pods from Services, but only if you tell it what "failed" and "ready" mean. It can also show a lot about what is going on, if you set up metrics and logs. Probes, metrics and logs are the sensory system of a cluster, and wrong settings here cause some of the most confusing outages.

What you will learn

- The three probes and what each should check
- Probe settings that avoid restart storms
- Resource metrics, the metrics server and what autoscaling uses
- Application metrics and Prometheus scraping
- Logging patterns and what to collect

The three probes

Probe	Question	On failure
Startup	Has the app finished starting?	Keeps other probes off; restarts if it never succeeds
Readiness	Can it serve traffic now?	Removes the Pod from Service endpoints, no restart
Liveness	Is it stuck for good?	Restarts the container

```
containers:
  - name: api
    image: registry.example.com/api:1.8.2
    ports: [{ containerPort: 8080 }]
    startupProbe:
      httpGet: { path: /healthz, port: 8080 }
      periodSeconds: 5
      failureThreshold: 30        # up to 150 seconds to start
    readinessProbe:
      httpGet: { path: /ready, port: 8080 }
      periodSeconds: 5
      timeoutSeconds: 2
      failureThreshold: 3
    livenessProbe:
      httpGet: { path: /healthz, port: 8080 }
      periodSeconds: 10
      timeoutSeconds: 2
      failureThreshold: 3
    resources:
      requests: { cpu: 200m, memory: 256Mi }
      limits: { memory: 512Mi }
```

Rules that prevent trouble:

- A liveness probe should check only the process itself (is the event loop alive?), never a dependency. If the database is down and liveness fails, every Pod restarts at once and the outage gets worse.
- Readiness may check dependencies the app cannot work without, but be careful: if all Pods become unready together, the Service has no endpoints.
- Use a startup probe for slow starters instead of a long `initialDelaySeconds`.
- Set timeouts shorter than the period, and thresholds that tolerate a short delay.

Metrics

The **metrics server** gives `kubectl top` and feeds the horizontal Pod autoscaler with CPU and memory use. It is not a monitoring system and keeps no history. For history and alerts, run Prometheus (or a compatible service) that scrapes:

- node and Pod resource use (cAdvisor, kubelet)
- cluster object state (`kube-state-metrics`: Pod phase, restarts, Deployment replicas)
- your applications, which expose `/metrics` and are discovered through annotations or `ServiceMonitor` objects

```
$ kubectl top pods -n shop --sort-by=memory
NAME           CPU(cores)   MEMORY(bytes)
api-7d9f-abc   180m         410Mi
$ kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes | head -c 200
```

Useful alerts: Pods restarting repeatedly, containers near their memory limit, Deployments with fewer available replicas than desired, nodes not ready, and persistent volumes nearly full.

Logging

Applications write to standard output, and the node keeps the files under `/var/log/containers`. Those files are rotated and disappear with the Pod, so run a node-level agent (a DaemonSet such as Fluent Bit or Vector) that ships logs to central storage with labels for namespace, Pod and container. Also collect cluster events, because `kubectl get events` forgets after about an hour.

```
kubectl logs deploy/api --since=15m --all-containers
kubectl logs api-7d9f-abc --previous
kubectl get events -A --sort-by=.lastTimestamp | tail -20
```

Write structured logs with a request or trace id, and avoid logging secrets and personal data.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Probes, metrics and logging on Kubernetes fits together

Part	What it does	What goes wrong
The three probes	Rules that prevent trouble.	Liveness probes that check dependencies.
Metrics	The metrics server gives kubectl top and feeds the horizontal Pod autoscaler with CPU and memory use.	Treating the metrics server as monitoring.
Logging	Applications write to standard output, and the node keeps the files under /var/log/containers.	No readiness probe, so traffic goes to Pods still starting.

Flow: The three probes to Metrics / Metrics to Logging

Incident: the restart storm

> **Incident:** *A short database failover causes every API Pod to restart, and the service stays down for ten minutes after the database is back.*

```
$ kubectl get pods -l app=api
api-7d9f-abc   0/1   Running   9 (20s ago)
$ kubectl describe pod api-7d9f-abc | grep -A2 Liveness
Liveness probe failed: HTTP probe failed with statuscode: 500
```

The liveness endpoint ran a database query. During failover it failed, all Pods were killed, and they then started together and overloaded the database. The team changed liveness to a simple process check, kept the database check in readiness with a lenient threshold, added a startup probe, and alerted on restart rate.

MARGA by EknathaLabs

Common mistakes

- Liveness probes that check dependencies
- No readiness probe, so traffic goes to Pods still starting
- Treating the metrics server as monitoring
- Only collecting logs from Pods that still exist

Check yourself

What happens to a Pod that fails its readiness probe?

Answer: It is removed from Service endpoints until it passes

Why should a liveness probe avoid checking the database?

Answer: A database failure would restart every Pod and make recovery harder

What is the metrics server used for?

Answer: Current CPU and memory data for kubectl top and autoscaling

Why run a log agent as a DaemonSet?

Answer: One agent per node can collect logs from every Pod on that node

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 17. Service mesh, mutual TLS and traffic control

14 minutes. Understand what a service mesh adds to a cluster, how sidecar and sidecarless data planes work, how mutual TLS and traffic policies are applied, and when a mesh is not worth its cost.

As services multiply, the same needs appear in all of them: encrypt traffic between services, know who is calling whom, retry failed calls sensibly, and shift traffic gradually between versions. A service mesh moves these concerns out of application code and into the platform. It is powerful and it is costly, so you should know both sides.

What you will learn

- What a mesh adds: identity, encryption, traffic control and telemetry
- Data plane and control plane
- Mutual TLS and authorization policy
- Traffic splitting, retries and timeouts
- The cost: complexity, resources and debugging

Parts of a mesh

The **data plane** is a set of proxies that handle service-to-service traffic. In the sidecar model each Pod gets a proxy container (often Envoy); newer designs use a per-node proxy or an eBPF-based approach to avoid a sidecar in every Pod. The **control plane** (Istio, Linkerd, Cilium service mesh and others) configures the proxies, issues certificates and collects telemetry.

Applications keep making plain calls to `http://payments`; the proxy intercepts and adds behaviour.

Mutual TLS and identity

With mutual TLS (mTLS) both sides present certificates, so each service knows who is calling and the traffic is encrypted. The mesh gives each workload an identity (for example, based on its Kubernetes service account) and rotates short-lived certificates automatically. You get encryption between services without changing code.

On top of identity you can write authorization rules: only the `checkout` workload may call `POST /charge` on `payments`.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: payments-allow-checkout, namespace: shop }
spec:
  selector: { matchLabels: { app: payments } }
  action: ALLOW
  rules:
    - from: [{ source: { principals: ["cluster.local/ns/shop/sa/checkout"] } }]
      to:   [{ operation: { methods: ["POST"], paths: ["/charge"] } }]
```

Roll out mTLS in permissive mode first (accept both plain and mTLS), check that nothing still sends plain traffic, then switch to strict.

MARGA by EknathaLabs marga.eknathalabs.com

Traffic control

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: reviews }
spec:
  hosts: [reviews]
  http:
    - route:
        - destination: { host: reviews, subset: v1 }
          weight: 90
        - destination: { host: reviews, subset: v2 }
          weight: 10
      timeout: 2s
      retries: { attempts: 2, perTryTimeout: 500ms, retryOn: "5xx,connect-failure" }
```

This sends 10 percent of traffic to version 2, with a timeout and limited retries. Be careful with retries: they multiply load during an outage (a retry storm) and are only safe for requests that can be repeated. Circuit breaking and outlier detection remove failing instances from rotation.

Telemetry

Because proxies see every call, the mesh can report request rate, errors and latency per service pair without code changes, and can create trace spans. Applications must still pass trace headers along for full traces.

The cost

- Extra resources: each sidecar uses CPU and memory, and adds a little latency.
- More moving parts to upgrade, and a new class of failures (certificate expiry, proxy misconfiguration).
- Debugging has another layer: a 503 may come from the proxy, not the app.
- Skills: the team must understand the mesh.

A mesh pays off with many services, strict security needs (mTLS everywhere, fine-grained policy) or frequent progressive delivery. For a handful of services, simpler tools may be enough: network policies, an ingress controller, and application-level TLS.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Service mesh, mutual TLS and traffic control fits together

Part	What it does	What goes wrong
Parts of a mesh	The data plane is a set of proxies that handle service-to-service traffic.	No alert on mesh certificate expiry.
Mutual TLS and identity	With mutual TLS (mTLS) both sides present certificates, so each service knows who is calling and the traffic is encrypted.	Enabling strict mTLS without migration.
Traffic control	This sends 10 percent of traffic to version 2, with a timeout and limited retries.	Unlimited retries.
Telemetry	Because proxies see every call, the mesh can report request rate, errors and latency per service pair without code changes, and can create trace spans.	Adopting a mesh for two services.
The cost	Extra resources: each sidecar uses CPU and memory, and adds a little latency.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Parts of a mesh to Mutual TLS and identity / Mutual TLS and identity to Traffic control / Traffic control to Telemetry / Telemetry to The cost

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the certificate that expired inside the mesh

> **Incident:** *At 02:10 many internal calls fail with 503 and "upstream connect error", but applications look healthy.*

```
$ istioctl proxy-status | head -3
NAME                CLUSTER  CDS     LDS     EDS     RDS
api-7d9f.shop       prod     STALE   STALE   STALE   STALE
$ kubectl logs deploy/istiod -n istio-system --since=1h | grep -i cert
x509: certificate has expired
```

The mesh's root certificate had expired because rotation had been left manual. Proxies could not get new configuration or certificates. The team issued a new root, rolled the control plane and proxies, and then automated rotation with an alert on certificate expiry 30 days ahead.

Common mistakes

- Enabling strict mTLS without migration
- Unlimited retries
- No alert on mesh certificate expiry
- Adopting a mesh for two services

Check yourself

What does mutual TLS add compared with ordinary TLS?

Answer: Both client and server prove their identity with certificates

What is the control plane responsible for?

Answer: Configuring proxies, issuing certificates and collecting telemetry

Why can retries be dangerous?

Answer: They multiply load during an outage and may repeat non-repeatable requests

When might a mesh be more than you need?

Answer: For a few services, where network policies and ingress already meet the need

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 18. Network policies in practice

13 minutes. Restrict which pods can talk to which, starting from default deny, and test the rules.

By default every pod in a Kubernetes cluster can talk to every other pod. A compromised pod in one namespace can reach the database in another. Network policies are the firewall rules of the cluster. They are simple, but they are easy to get wrong in ways that either break the application or protect nothing.

What you will learn

- The default is open, and what a NetworkPolicy changes
- Default deny for ingress and egress
- Allow rules with pod selectors, namespace selectors and ports
- Test policies, and remember DNS

What a policy does

A NetworkPolicy selects pods with labels and says which traffic is allowed to or from them. Two facts matter first. Policies are **additive allow rules**: once any policy selects a pod for a direction, only traffic that some policy allows is permitted in that direction. And policies are enforced by the **network plugin**; if your plugin does not support them, the objects exist and do nothing. Calico, Cilium and several cloud plugins support them.

Start with default deny

Begin each namespace with a policy that denies all ingress to all pods, then add what is needed.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Ingress"]
```

The empty `podSelector` selects every pod in the namespace. With no ingress rules listed, nothing is allowed in. Do the same for egress later, once you know what each workload needs to reach.

Allow what is needed

Add policies that name the exact paths. This one lets only the `api` pods connect to the `db` pods on the database port:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-from-api
  namespace: shop
spec:
  podSelector:
    matchLabels: { app: db }
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: api }
      ports:
        - { protocol: TCP, port: 5432 }
```

To allow traffic from another namespace, use a `namespaceSelector` (namespaces need labels, for example `kubernetes.io/metadata.name`). Be careful: putting `podSelector` and `namespaceSelector` in the same list item means *both must match*, while two separate items mean *either*.

MARGA by EknathaLabs marga.eknathalabs.com

Egress, DNS and testing

If you restrict egress, remember DNS. Pods must reach the cluster DNS service on port 53 (UDP and TCP), or every name lookup fails and the application looks broken for no visible reason.

```
$ kubectl -n shop run probe --rm -it --image=busybox:1.36 --restart=Never -- \
    wget -qO- -T 3 http://db:5432
$ kubectl -n shop get networkpolicy
$ kubectl -n shop describe networkpolicy db-from-api
```

Test both directions: the path that must work, and a path that must be blocked. Keep policies in version control next to the application manifests, and review them in the same pull request as the workload change.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Network policies in practice fits together

Part	What it does	What goes wrong
What a policy does	A NetworkPolicy selects pods with labels and says which traffic is allowed to or from them.	Assuming policies work without a plugin that enforces them.
Start with default deny	Begin each namespace with a policy that denies all ingress to all pods, then add what is needed.	Forgetting DNS when restricting egress.
Allow what is needed	Add policies that name the exact paths.	Writing policies once and never testing the blocked paths.
Egress, DNS and testing	If you restrict egress, remember DNS. Pods must reach the cluster DNS service on port 53 (UDP and TCP), or every name lookup fails and the application looks broken for...	Mixing up selector lists, so a rule is far wider than intended.

Flow: What a policy does to Start with default deny / Start with default deny to Allow what is needed / Allow what is needed to Egress, DNS and testing

Incident: Lateral movement in one hop

> **Incident:** *A vulnerable image in the* `marketing` *namespace was exploited, and the attacker connected straight to the payments database.*

```
$ kubectl get networkpolicy -A
No resources found
$ kubectl -n marketing exec web-6d9 -- nc -zv payments-db.payments 5432
payments-db.payments (10.96.14.8:5432) open
```

The cluster had no network policies, so any pod could reach any other. The incident review led to default deny policies in every namespace, explicit allow rules for the payments database from the payments namespace only, and an admission policy that rejects a new namespace until it has a default deny. A weekly test job tries a blocked connection from a canary pod and alerts if it succeeds.

Common mistakes

- Assuming policies work without a plugin that enforces them
- Forgetting DNS when restricting egress
- Mixing up selector lists, so a rule is far wider than intended
- Writing policies once and never testing the blocked paths

Check yourself

What is the default behaviour without any NetworkPolicy?

Answer: All pods can talk to all other pods

MARGA by EknathaLabs marga.eknathalabs.com

Which object enforces NetworkPolicy rules?

Answer: The network plugin (CNI), such as Calico or Cilium

What does an empty podSelector select?

Answer: Every pod in the namespace

Why test a blocked path?

Answer: A rule that blocks nothing looks the same as one that works until you try

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 19. StatefulSets and stateful workloads

14 minutes. Run workloads that need stable names and their own storage, and decide honestly when not to run a database on Kubernetes.

A Deployment treats pods as interchangeable. A database node is not interchangeable: it has its own data, a name the others know it by, and a role (primary or replica). StatefulSets give pods a stable identity and storage, but they do not make a database safe by themselves. This lesson covers what they provide and what you still have to do.

What you will learn

- What a StatefulSet guarantees: names, order and storage
- Headless services and stable DNS names
- Volume claim templates and what happens when a pod moves
- When to use an operator or a managed database instead

Stable identity

A StatefulSet creates pods named `name-0`, `name-1`, `name-2`. If `db-1` is deleted, the replacement is again called `db-1` and gets the same storage. By default pods start in order (0 first) and stop in reverse order, which suits clusters where the first node bootstraps the others.

```
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: db }
spec:
  serviceName: db-headless
  replicas: 3
  selector: { matchLabels: { app: db } }
  template:
    metadata: { labels: { app: db } }
    spec:
      containers:
        - name: db
          image: postgres:16
          volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
  volumeClaimTemplates:
    - metadata: { name: data }
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: fast
        resources: { requests: { storage: 100Gi } }
```

Headless service and DNS

A **headless service** (`clusterIP: None`) does not load balance. Instead, DNS returns a record per pod: `db-0.db-headless.shop.svc.cluster.local`. Peers and clients can therefore reach one specific member, for example the primary at `db-0`. Use a normal service in front for clients that need any healthy node.

Storage and what happens on failure

Each pod gets its own PersistentVolumeClaim from the `volumeClaimTemplates`. If the pod is deleted or rescheduled, the new pod attaches the same claim. Note two things:

- Deleting a StatefulSet does **not** delete the claims, so data survives, and you must remove them deliberately. Newer versions allow a retention policy for this.
- A volume is tied to a zone in most clouds. If the zone fails, the pod cannot start elsewhere without that data, so replication across zones is the database's job, not Kubernetes'.

A PodDisruptionBudget stops a node drain from taking down too many members at once, for example `maxUnavailable: 1`.

MARGA by EknathaLabs marga.eknathalabs.com

Be honest about databases

StatefulSets give identity and storage. They do not give you backups, failover, upgrades, tuning or point-in-time recovery. For databases you have three choices:

1. A **managed service** from your cloud: least work, usually the right default for important data.
2. A Kubernetes **operator** for that database, which encodes failover, backups and upgrades as automation. Use a well-maintained one, and test restores.
3. A hand-written StatefulSet: only for learning or simple, replaceable data such as a cache.

Whichever you choose, rehearse: kill the primary, restore from backup into a fresh cluster, and measure how long it takes.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How StatefulSets and stateful workloads fits together

Part	What it does	What goes wrong
Stable identity	A StatefulSet creates pods named name-0, name-1, name-2.	Treating a StatefulSet as a complete database solution.
Headless service and DNS	A headless service (clusterIP: None) does not load balance.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Storage and what happens on failure	Each pod gets its own PersistentVolumeClaim from the volumeClaimTemplates.	Assuming storage can follow a pod to another zone.
Be honest about databases	StatefulSets give identity and storage. They do not give you backups, failover, upgrades, tuning or point-in-time recovery.	Forgetting backups and restore tests.

Flow: Stable identity to Headless service and DNS / Headless service and DNS to Storage and what happens on failure / Storage and what happens on failure to Be honest about databases

Incident: The replica that came back empty

> **Incident:** *After a node failure, a database replica restarted and began a full resync that took six hours and slowed the primary.*

```
$ kubectl get pvc -l app=db
NAME        STATUS   VOLUME    CAPACITY
data-db-0   Bound    pvc-a1    100Gi
data-db-1   Pending  <none>    100Gi
```

The volume for `db-1` lived in a zone that was down, and the claim could not be bound elsewhere, so the replacement pod started with a new empty volume created by the storage class. The team switched to an operator with zone-aware replicas, added a PodDisruptionBudget, and scheduled a monthly restore drill. They also moved the main production database to a managed service, and kept the StatefulSet for a rebuildable analytics store.

Common mistakes

- Treating a StatefulSet as a complete database solution
- Forgetting backups and restore tests
- Assuming storage can follow a pod to another zone
- Deleting claims by accident while cleaning up

Check yourself

What does a StatefulSet give each pod?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: A stable name and its own persistent volume

What is a headless service used for?

Answer: DNS records for individual pods, not load balancing

What happens to PVCs when a StatefulSet is deleted by default?

Answer: They stay until you delete them

Which option usually needs the least operational work for an important database?

Answer: A managed database service

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 20. Debugging Kubernetes workloads

18 minutes. Follow a fixed path from Pending to CrashLoopBackOff, ImagePullBackOff, failing probes and unreachable Services using describe, logs, events, exec and ephemeral debug containers.

Kubernetes reports problems in a small set of states: Pending, ImagePullBackOff, CrashLoopBackOff, Running but not Ready, and a Service that routes nowhere. Each state points to a different part of the system. Learn the state, then the one or two commands that explain it.

What you will learn

- What each Pod state means and where to look
- `describe`, `logs --previous`, `get events`, `exec` and `kubectl debug`
- Checking Services, endpoints, DNS and network policy
- Node-level problems: pressure, taints and capacity

Start with the Pod

```
$ kubectl get pods -o wide
NAME            READY   STATUS             RESTARTS   AGE
shop-7d9f-abc   0/1     CrashLoopBackOff   6          9m
$ kubectl describe pod shop-7d9f-abc | sed -n '/State:/,/Ready:/p;/Events:/,$p'
$ kubectl logs shop-7d9f-abc --previous --tail=50
$ kubectl get events --sort-by=.lastTimestamp -n shop | tail -15
```

`describe` shows the last state, exit code and events. `logs --previous` shows the output of the container that just died, which `logs` alone does not.

State	Meaning	First check
Pending	Not scheduled	Events: insufficient CPU or memory, taints, unbound volume claim
ImagePullBackOff	Cannot pull the image	Image name and tag, registry secret, network
CrashLoopBackOff	Starts and exits repeatedly	`logs --previous`, exit code, OOMKilled, config
Running, not Ready	Readiness probe fails	Probe path and port, app startup time
Evicted	Node under pressure	Node conditions, resource requests
Terminating forever	Finalizer or stuck volume	`get pod -o yaml` for finalizers

Get inside, even without a shell

```
kubectl exec -it shop-7d9f-abc -- sh
kubectl debug -it shop-7d9f-abc --image=busybox:1.36 --target=app   # ephemeral container
kubectl debug node/worker-2 -it --image=busybox:1.36                # look at a node
kubectl get pod shop-7d9f-abc -o yaml | less                         # the full spec and status
```

An ephemeral container shares the Pod's namespaces, so you can test with tools the production image does not ship.

A Service that goes nowhere

```
$ kubectl get svc shop -o wide
$ kubectl get endpoints shop
NAME   ENDPOINTS
shop   <none>
$ kubectl get pods --show-labels
```

Empty endpoints mean the Service selector matches no Ready Pod. Compare the selector with the Pod labels and check readiness. Then test from inside the cluster:

```
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://shop.shop.svc.cl \
    uster.local
```

MARGA by EknathaLabs marga.eknathalabs.com

If the name does not resolve, check CoreDNS Pods. If it resolves but hangs, check NetworkPolicy and the target port.

Diagram: From a broken Pod to a cause

Part	What it does	What goes wrong
Pod state	kubectl get pods shows Pending, CrashLoopBackOff or not Ready.	The state alone does not give the cause.
Describe and events	Shows exit code, scheduling messages, probe failures and pull errors.	Events expire after about an hour, so look soon.
Logs, previous run	logs --previous prints output from the crashed container.	Plain logs show only the new, empty container.
Service and endpoints	Empty endpoints mean the selector or readiness is wrong.	A correct Pod behind a mismatched selector still receives no traffic.
Node and policy	Check node conditions, taints, quotas and NetworkPolicy.	The Pod can be fine while the node or policy blocks it.

Flow: Pod state to Describe and events / Pod state to Logs, previous run / Describe and events to Service and endpoints / Logs, previous run to Node and policy

Incident: healthy Pods, no traffic

Incident: *After a rollout, the shop returns 503 from the ingress although all Pods show Running.*

```
$ kubectl get pods
shop-8c4d-x2k   0/1   Running   0   4m
$ kubectl describe pod shop-8c4d-x2k | grep -i -A2 readiness
Readiness probe failed: Get "http://10.2.1.14:8080/healthz": connect: connection refused
$ kubectl get cm shop-config -o yaml | grep PORT
PORT: "9090"
```

A config change moved the application to port 9090, but the readiness probe still checked 8080. No Pod became Ready, so the Service had no endpoints. The fix was to align the probe and the config, and the team added a rollout check that fails if the new ReplicaSet does not become Ready.

Common mistakes

- Reading `logs` instead of `logs --previous` after a crash
- Assuming Running means working
- Forgetting events disappear
- Editing Pods directly instead of the Deployment

Check yourself

A Pod is in CrashLoopBackOff. Which command shows why the last container died?

Answer: kubectl logs POD --previous

kubectl get endpoints shows <none> for a Service. What is the usual cause?

Answer: The selector matches no Ready Pods

A Pod is Pending with "Insufficient memory". What does the scheduler compare?

Answer: The Pod's memory request against free allocatable memory on nodes

How do you inspect a Pod whose image has no shell?

Answer: Attach an ephemeral container with kubectl debug

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Kubernetes

kubectl essentials, manifests, troubleshooting order, RBAC and exam-speed tricks.

Setup for speed

```
alias k=kubectl
source <(kubectl completion bash)
export do="--dry-run=client -o yaml"
k config get-contexts
k config use-context CONTEXT
k config set-context --current --namespace=shop
```

Always confirm the context with `k config current-context` before changing anything.

Read cluster state

```
k get nodes -o wide
k get pods -A
k get pods -n shop -o wide --show-labels
k get deploy,svc,ingress,pvc -n shop
k get events -n shop --sort-by=.lastTimestamp
k describe pod NAME -n shop
k explain pod.spec.containers.resources
k api-resources
k top pod -n shop
```

Create and change

```
k create deployment api --image=nginx:1.27 --replicas=3 $do > deploy.yaml
k run tmp --rm -it --image=busybox:1.36 --restart=Never -- sh
k expose deployment api --port=80 --target-port=8080
k scale deployment api --replicas=5
k set image deployment/api nginx=nginx:1.27.1
k apply -f deploy.yaml
k delete -f deploy.yaml
k label pod NAME tier=backend
k annotate pod NAME note=checked
k edit deployment api
```

Rollouts

```
k rollout status deployment/api
k rollout history deployment/api
k rollout undo deployment/api
k rollout undo deployment/api --to-revision=2
k rollout restart deployment/api
```

Debugging order

```
k get pods -n shop                      # status and restarts
k describe pod NAME -n shop             # events at the bottom
k logs NAME -n shop --previous          # logs of the crashed container
k logs NAME -c CONTAINER -n shop        # a specific container
k exec -it NAME -n shop -- sh
k get endpoints SERVICE -n shop         # empty means selector or readiness problem
k debug NAME -it --image=busybox:1.36   # ephemeral debug container
```

Status	Likely cause
`Pending`	No node fits, or an unbound PVC
`ImagePullBackOff`	Wrong image or missing registry credentials
`CrashLoopBackOff`	Application error; read `--previous` logs
`CreateContainerConfigError`	Missing ConfigMap, Secret or key
`OOMKilled`	Memory limit exceeded
Running, `0/1` ready	Readiness probe failing

MARGA by EknathaLabs marga.eknathalabs.com

Pod template essentials

```
resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits: { memory: 256Mi }
readinessProbe:
  httpGet: { path: /healthz/ready, port: 8080 }
livenessProbe:
  httpGet: { path: /healthz/live, port: 8080 }
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: { drop: ["ALL"] }
```

Services

Type	Reachable from
`ClusterIP`	Inside the cluster
`NodePort`	Node IP on a high port
`LoadBalancer`	Cloud load balancer address
`ExternalName`	DNS alias for an external name

Service DNS name: `SERVICE.NAMESPACE.svc.cluster.local`.

RBAC

```
k create role reader --verb=get,list,watch --resource=pods -n shop
k create rolebinding rb --role=reader --serviceaccount=shop:app -n shop
k auth can-i list pods -n shop --as=system:serviceaccount:shop:app
k auth can-i --list -n shop
```

Nodes and maintenance

```
k cordon NODE                                  # stop new scheduling
k drain NODE --ignore-daemonsets --delete-emptydir-data
k uncordon NODE
k taint nodes NODE dedicated=gpu:NoSchedule
k label node NODE disktype=ssd
```

Storage

Item	Note
PVC ReadWriteOnce	One node at a time
PVC ReadWriteMany	Needs a file-capable storage class
Reclaim Delete	Claim deletion deletes the disk
StatefulSet	Stable names and a volume per Pod

JSONPath and output

```
k get pods -o jsonpath='{.items[*].metadata.name}'
k get pod NAME -o jsonpath='{.status.containerStatuses[0].lastState}'
k get secret NAME -o jsonpath='{.data.password}' | base64 -d
k get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName
```

Context and namespaces

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context staging
kubectl config set-context --current --namespace=shop
kubectl get ns
kubectl api-resources --namespaced=true
kubectl explain deployment.spec.strategy
kubectl auth can-i delete pods -n shop
kubectl auth can-i --list -n shop
```

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Check* `kubectl config current-context` *before any change. Most production accidents are the right command on the wrong cluster. Consider a prompt that shows the context.*

Inspect and debug, more

```
kubectl get pods -A -o wide --field-selector status.phase!=Running
kubectl get events -n shop --sort-by=.lastTimestamp
kubectl describe pod api-0 -n shop
kubectl logs api-0 -c app --previous --tail=200
kubectl logs -l app=api --all-containers --since=15m
kubectl exec -it api-0 -c app -- sh
kubectl debug -it api-0 --image=busybox:1.36 --target=app
kubectl debug node/worker-1 -it --image=busybox:1.36
kubectl port-forward svc/api 8080:80 -n shop
kubectl cp shop/api-0:/tmp/dump.txt ./dump.txt
kubectl top pod -n shop --containers
kubectl top node
kubectl get pod api-0 -o yaml | less
```

Warning: `kubectl exec` *and* `debug` *give shell access to a running workload. Do not change files or install tools in a production container: the change is lost and may hide the real cause.*

Warning: `kubectl port-forward` *exposes the service to your machine only, but anything you paste through it goes to production. Never point a test tool at it.*

Change safely

```
kubectl diff -f deploy.yaml
kubectl apply -f deploy.yaml --dry-run=server
kubectl apply -f deploy.yaml
kubectl rollout status deploy/api -n shop
kubectl rollout undo deploy/api -n shop
kubectl scale deploy/api --replicas=5 -n shop
kubectl set image deploy/api app=registry.example.com/api:1.5 -n shop
kubectl annotate deploy/api kubernetes.io/change-cause="bump to 1.5"
kubectl label pod api-0 debug=true
kubectl patch deploy api -p '{"spec":{"replicas":3}}'
```

Warning: `kubectl edit` *and* `patch` *change live objects that are not in your source files, so the next* `apply` *or pipeline run can undo them silently, or the cluster can drift. Prefer changing the manifest and applying it.*

Warning: `kubectl scale --replicas=0` *stops the service. Do it only on purpose and say so in the incident channel.*

Nodes and maintenance, more

```
kubectl get nodes -o wide
kubectl describe node worker-1 | sed -n '/Taints/,/Conditions/p'
kubectl cordon worker-1
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data --timeout=300s
kubectl uncordon worker-1
kubectl taint nodes worker-1 dedicated=gpu:NoSchedule
kubectl get pdb -A
```

Danger: `kubectl drain` *evicts pods. With* `--delete-emptydir-data` *the data in emptyDir volumes is lost, and a PodDisruptionBudget of 0 can block it forever. Drain one node at a time and watch the rollout.*

Danger: `kubectl delete node` *removes the node object, not the machine. Drain first and confirm the cloud instance state.*

Secrets and config

```
kubectl create secret generic db --from-literal=password='change-me' -n shop --dry-run=client -o y \
    aml
kubectl get secret db -n shop -o jsonpath='{.data.password}' | base64 -d
kubectl create configmap app-config --from-file=app.properties
kubectl get cm app-config -o yaml
kubectl rollout restart deploy/api -n shop
```

Warning: *Base64 is not encryption. Anyone who can read a Secret can read its value. Restrict access with RBAC, and do not paste decoded values into chat or tickets.*

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Passwords on the command line end up in shell history. Prefer files or a secrets manager.*

Cautions for destructive commands

Danger: `kubectl delete -f` *and* `kubectl delete ns shop` *remove everything in the file or namespace, including volumes with* `Delete` *reclaim policy. Run* `kubectl get all -n shop` *and use* `--dry-run=server` *first.*

Danger: `kubectl delete pod --force --grace-period=0` *skips graceful shutdown and can leave a StatefulSet pod running twice. Use it only for stuck terminating pods after you confirm the node is gone.*

Danger: `kubectl delete pvc` *can delete the underlying disk. Check* `kubectl get pv` *for the reclaim policy first.*

Danger: *Commands with* `-A` *or* `--all` *act on every namespace. Read the output of the matching* `get` *before you run* `delete`*.*

Warning: *Do not bind* `cluster-admin` *to users or service accounts for convenience. Grant the smallest Role that works.*

MARGA by EknathaLabs
marga.eknathalabs.com